

UNIVERSIDAD NACIONAL DE TUCUMÁN  
FACULTAD DE CIENCIAS EXACTAS Y TECNOLOGÍA  
DEPARTAMENTO DE ELECTRICIDAD, ELECTRÓNICA Y COMPUTACIÓN



TRABAJO DE GRADUACIÓN, INFORME FINAL

Diseño de un ADC SAR de 12 bits @ 40 MSa/s en Tecnología ihp-sg13g2

Jesus Gerardo Daniel Avila  
DNI 41650372  
Ing Electrónica

Tutores: Miranda Bonomi, Fernando Scandaliaris, Jorge

Diciembre, 2025

## **ANEXO III - FICHA DE PRESENTACIÓN DE PROYECTOS FINALES**

**NOMBRE DEL PROYECTO:** Diseño de un ADC SAR de 12 bits @ 40 MSa/s en Tecnología ihp-sg13g2

**APELLIDO(S) Y NOMBRE(S) DEL (DE LOS) ALUMNO(S):** Jesus Gerardo Daniel Avila

**DNI:** 41650372

**OBJETIVOS:** Tener un diseño de ejemplo en herramientas Open-Source para la catedra de Diseño de Circuitos Integrados

**ESPECIFICACIONES:** 12 Bits @40MSa/s

**CONCEPTOS TEÓRICOS PRÁCTICOS INVOLUCRADOS:** Diseño VLSI custom

**DURACIÓN:** 7 meses

**LUGAR DE REALIZACIÓN:** Laboratorio Telecomunicaciones (LabTel) - FACET

**RESPONSABLE O TUTOR: Miranda Bonomi, Fernando**

Firma:

Fecha:

**RESPONSABLE O TUTOR: Scandaliaris, Jorge**

Firma:

Fecha:

# Dedicatoria

*A mi Padre, por quien empecé esta carrera  
A mi Madre, por quien puedo terminarla*

# Resumen

Este trabajo presenta el diseño y análisis de un convertidor analógico-digital (ADC) de aproximaciones sucesivas (SAR) de 12 bits operando a 40 MSa/s en tecnología ihp-sg13g2. El diseño implementa un CDAC con arquitectura Inverted Merged Capacitor Switching (IMCS) basada en modo común, un comparador de doble cola tipo Bindra, y lógica SAR síncrona realizada con celdas estándar. Se documentan decisiones de diseño, simulaciones y caracterización del sistema, con énfasis en aspectos didácticos y uso de herramientas de código abierto.

# Índice general

|                                                                                       |           |
|---------------------------------------------------------------------------------------|-----------|
| <b>Resumen</b>                                                                        | <b>5</b>  |
| <b>1. Introducción</b>                                                                | <b>8</b>  |
| 1.1. Motivación . . . . .                                                             | 8         |
| 1.2. Estado del Arte . . . . .                                                        | 8         |
| 1.2.1. Evolución de los Comparadores . . . . .                                        | 8         |
| 1.2.2. Arquitecturas de CDAC . . . . .                                                | 9         |
| 1.2.3. Implementaciones Digitales . . . . .                                           | 9         |
| 1.3. Resumen del Trabajo . . . . .                                                    | 9         |
| 1.3.1. Alcance y Limitaciones . . . . .                                               | 10        |
| 1.4. Requerimientos del Sistema . . . . .                                             | 10        |
| 1.5. Organización del documento . . . . .                                             | 10        |
| <b>2. Implementación del SAR ADC</b>                                                  | <b>12</b> |
| 2.1. Descripción del SAR ADC nivel TOP . . . . .                                      | 13        |
| 2.1.1. Arquitectura Inverted Merged Capacitor Switching (IMCS) . . . . .              | 14        |
| 2.2. DAC Capacitivo (CDAC) . . . . .                                                  | 16        |
| 2.2.1. Definición del Capacitor Unitario ( $C_u$ ) . . . . .                          | 16        |
| 2.2.2. Implementación del Array Split-Binary . . . . .                                | 16        |
| 2.2.3. Cálculo de Capacitores de Atenuación ( $C_B$ ) y Grounding ( $C_X$ ) . . . . . | 17        |
| 2.2.4. Lógica de las señales de control de la Bottom Plate de cada celda . . . . .    | 18        |
| 2.2.5. Generador de Señales Non-Overlap . . . . .                                     | 20        |
| 2.2.6. Top Plate Sample switch . . . . .                                              | 22        |
| 2.2.7. Requerimiento de Settling . . . . .                                            | 22        |
| 2.2.8. Dimensionamiento del Transmission Gate (sample) . . . . .                      | 23        |
| 2.2.9. Compensación de Inyección de Carga con Dummies . . . . .                       | 28        |
| 2.2.10. Testbench del CDAC . . . . .                                                  | 30        |
| 2.3. Comparador de Doble Cola . . . . .                                               | 32        |
| 2.3.1. Presupuesto de Tiempos . . . . .                                               | 32        |
| 2.3.2. Arquitectura del Comparador . . . . .                                          | 33        |
| 2.3.3. Testbench del Comparador . . . . .                                             | 34        |
| 2.3.4. Generador de Pulsos (Delay Line) . . . . .                                     | 37        |
| 2.4. Buffers de Control . . . . .                                                     | 40        |
| 2.5. Lógica SAR . . . . .                                                             | 45        |
| 2.5.1. Bloque Unitario de Lógica . . . . .                                            | 46        |
| 2.5.2. Implementación con Celdas Estándar . . . . .                                   | 47        |
| 2.5.3. Análisis de Timing . . . . .                                                   | 47        |
| 2.6. Simulación TOP . . . . .                                                         | 48        |

|                                                                       |           |
|-----------------------------------------------------------------------|-----------|
| 2.6.1. Caracterización . . . . .                                      | 50        |
| <b>3. Diseño de Layout</b>                                            | <b>52</b> |
| 3.1. Preamplificador . . . . .                                        | 53        |
| <b>4. Conclusiones y Trabajo Futuro</b>                               | <b>54</b> |
| 4.1. Estado Actual del Trabajo . . . . .                              | 54        |
| 4.2. Conclusiones . . . . .                                           | 54        |
| 4.2.1. Logros Principales . . . . .                                   | 54        |
| 4.2.2. Lecciones Aprendidas y Recomendaciones Metodológicas . . . . . | 55        |
| 4.3. Trabajo Futuro . . . . .                                         | 55        |
| 4.3.1. Mejoras en Metodología y Automatización . . . . .              | 55        |
| 4.3.2. Mejoras Específicas del ADC . . . . .                          | 56        |
| 4.3.3. Análisis Avanzados . . . . .                                   | 56        |
| 4.3.4. Mejoras de Layout . . . . .                                    | 57        |
| 4.3.5. Escalabilidad: Arquitectura Time-Interleaved . . . . .         | 57        |
| 4.4. Reflexión Final . . . . .                                        | 58        |

# Capítulo 1

## Introducción

### 1.1. Motivación

Los convertidores analógico-digitales (ADCs) de aproximaciones sucesivas (SAR) son ampliamente utilizados en aplicaciones que requieren eficiencia energética y resoluciones medias-altas. En las últimas décadas, el diseño de SAR ADCs ha experimentado mejoras significativas: se ha conseguido reducir el consumo de energía aproximadamente 7500 veces en un periodo de 15 años [2], principalmente gracias al avance hacia nodos tecnológicos más pequeños y al desarrollo de técnicas de conmutación más eficientes.

El presente trabajo documenta el diseño completo de un ADC SAR de 12 bits operando a una velocidad de muestreo mínima de 1 MSa/s, utilizando herramientas de código abierto y la tecnología ihp-sg13g2. El objetivo principal de este trabajo es servir como una primera aproximación al diseño de circuitos integrados, sirviendo como material de referencia para la asignatura de Diseño de Circuitos Integrados de la Facultad de Ciencias Exactas y Tecnología (FACET).

La elección del SAR ADC como caso de estudio se fundamenta en su complejidad media, que permite ilustrar las diferentes etapas del flujo de diseño analógico sin alcanzar la complejidad de sistemas como PLLs o transceptores completos. Este enfoque didáctico prioriza la documentación exhaustiva del proceso de diseño sobre la optimización extrema del rendimiento.

### 1.2. Estado del Arte

En un SAR ADC bien optimizado, el retardo [3] (delay) y el consumo de energía [2] se distribuyen de manera equilibrada entre los tres bloques principales: CDAC, comparador y lógica digital.

#### 1.2.1. Evolución de los Comparadores

El comparador es típicamente implementado con un ancho de banda relativamente pequeño para filtrar el ruido proveniente del CDAC. La arquitectura clásica consiste en un preamplificador seguido de un latch que utiliza realimentación positiva para resolver la decisión. El preamplificador cumple la función de amplificar la señal de entrada y atenuar el offset, ruido y kickback del latch.

Desde la introducción del comparador Strong-ARM, se han propuesto diversas mejoras. Van Elzakker et al. [4] modificaron la topología del latch separando las etapas de

preamplificación y regeneración, asegurando que los transistores de entrada del latch comiencen en la región de saturación. Esto resulta en mayor ganancia y reducción del ruido y offset del comparador, mejorando simultáneamente la velocidad de operación.

Más recientemente, Bindra et al. [5] propusieron un comparador de polarización dinámica (dynamic bias) que alcanza un ruido de entrada de 0.4 mV operando a 1.2 V en tecnología de 65 nm CMOS, mejorando significativamente la eficiencia energética.

### 1.2.2. Arquitecturas de CDAC

Se han presentado diversas arquitecturas para el CDAC que mejoran sustancialmente la eficiencia energética respecto al esquema convencional [2]. Entre ellas destacan las arquitecturas basadas en modo común (VCM-based), que mantienen constante el voltaje de modo común durante la conversión, simplificando el diseño del comparador y mejorando la linealidad.

El esquema Inverted Merged Capacitor Switching (IMCS) [7] representa una evolución de estas técnicas, logrando eliminar la sensibilidad a capacitancias parásitas mientras mantiene una alta eficiencia energética (94 % menos consumo que el esquema convencional). Esta arquitectura invierte el proceso de muestreo: durante la fase de muestreo se captura  $V_{CM}$  en las placas superiores y la señal de entrada en las placas inferiores, asegurando que las placas superiores comiencen y terminen en el mismo voltaje.

### 1.2.3. Implementaciones Digitales

Respecto a la lógica de control, se han propuesto tanto diseños full-custom [3] como circuitos auto-sincronizados (self-clocked) para reducir el consumo asociado a la distribución de la señal de reloj. Estos últimos generan los pulsos de control basándose en la detección de la finalización de cada comparación, optimizando la velocidad de conversión.

## 1.3. Resumen del Trabajo

Este diseño está fuertemente basado en la arquitectura propuesta por Chang [7], adoptando el esquema de conmutación IMCS y el generador de pulsos auto-sincronizado. La implementación se diferencia en los siguientes aspectos:

- **Herramientas de Código Abierto:** Todo el flujo de diseño se realizó utilizando herramientas libres integradas en el entorno IIC-OSIC-TOOLS: xschem para captura esquemática, ngspice para simulación y klayout para layout. El proyecto completo está disponible en GitHub: [https://github.com/gsuckz/SAR\\_ADC\\_IHP](https://github.com/gsuckz/SAR_ADC_IHP).
- **Tecnología Abierta:** Se utiliza el PDK ihp-sg13g2, una tecnología de 130 nm completamente abierta y accesible para la comunidad académica.
- **Comparador de Polarización Dinámica:** Se implementó el comparador de doble cola propuesto por Bindra et al. [5], que incluye un capacitor en la cola del preamplificador para reducir el consumo de potencia.
- **Lógica con Celdas Estándar:** A diferencia de implementaciones full-custom, la lógica SAR se realizó con celdas estándar disponibles en el PDK, simplificando el diseño y facilitando la replicabilidad.

- **Enfoque Didáctico:** El trabajo prioriza la documentación exhaustiva del proceso de diseño sobre la optimización extrema del rendimiento, sirviendo como material educativo para estudiantes de circuitos integrados.

### 1.3.1. Alcance y Limitaciones

Este trabajo atraviesa las diferentes etapas del diseño de circuitos integrados: desde la captura esquemática y simulación hasta el diseño del layout final, verificando DRC (Design Rule Check) y LVS (Layout vs Schematic). Sin embargo, con el objetivo de mantener el enfoque didáctico y la complejidad manejable, se establecieron las siguientes limitaciones:

- No se realizaron simulaciones de mismatch (Monte Carlo) ni análisis de corners de proceso, voltaje y temperatura (PVT).
- No se implementaron técnicas avanzadas de optimización de parásitos en el layout.
- Las especificaciones mínimas establecidas son 12 bits de resolución y 1 MSa/s de velocidad de muestreo, sin restricciones específicas sobre otras métricas como SNDR, SFDR o consumo de potencia.

Debido a que no se realizaron simulaciones de ruido ni de variaciones de proceso (mismatch), la única especificación sobre la que se iteró es el tiempo de conversión, intentando reducirlo lo máximo posible. Se detallan los requerimientos perseguidos en la tabla 1.1

## 1.4. Requerimientos del Sistema

Cuadro 1.1: Requerimientos del ADC SAR

| Parámetro                            | Valor                 |
|--------------------------------------|-----------------------|
| Resolución                           | 12 bits (diferencial) |
| Velocidad de muestreo (mínima)       | 1 MSa/s               |
| Velocidad de muestreo (objetivo)     | 40 MSa/s              |
| Tensión de alimentación ( $V_{DD}$ ) | 1,8 V                 |
| Tensión de modo común ( $V_{CM}$ )   | 0,9 V                 |
| Tecnología                           | ihp-sg13g2 (130 nm)   |
| Arquitectura CDAC                    | IMCS con Split 6+6    |
| LSB                                  | 878 $\mu$ V           |
| Tiempo de Muestreo                   | 2 ns                  |

## 1.5. Organización del documento

- **Capítulo 2 - Implementación:** Se describe la implementación práctica de cada bloque del ADC SAR: CDAC con esquema IMCS, Switch de Sample & Hold, comparador de doble cola, lógica SAR y generador de pulsos.
- **Capítulo 3 - Layout:** Se adjuntan fotos del layout implementado y se explica la implementación del CDAC.

- **Capítulo 4 - Conclusiones:** Se resumen los resultados obtenidos, lecciones aprendidas y trabajo futuro.

Para fundamentos teóricos sobre arquitecturas SAR y esquemas de conmutación, se remite al lector a las referencias citadas [2, 7].

## Capítulo 2

# Implementación del SAR ADC

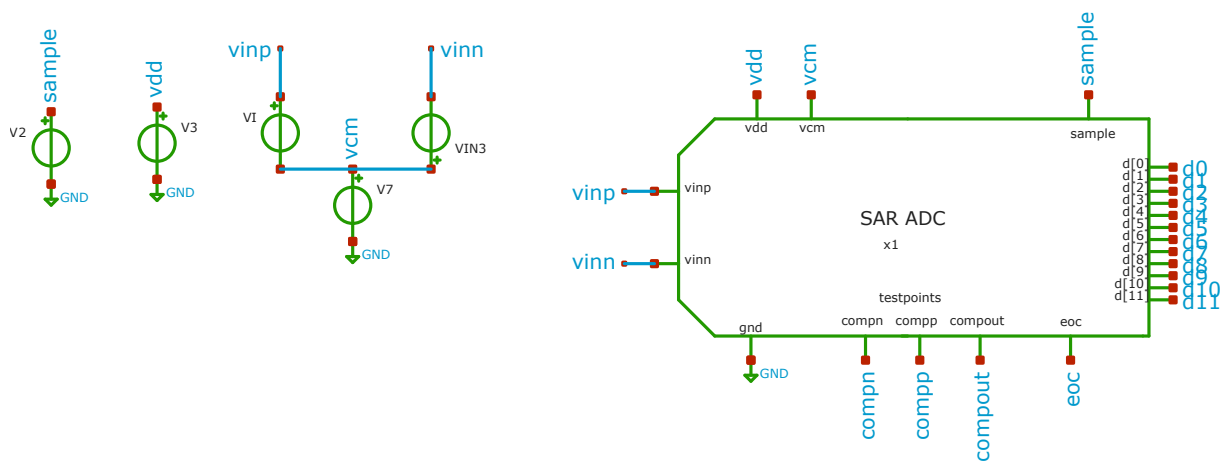


Figura 2.1: Testbench del SAR ADC

Este capítulo describe la implementación práctica de cada bloque del ADC SAR de 12 bits. Se detallan las decisiones de diseño, los testbenches utilizados para verificación y los resultados de simulación obtenidos. La la figura 2.2 muestra la interconexion de los bloques principales del sistema.

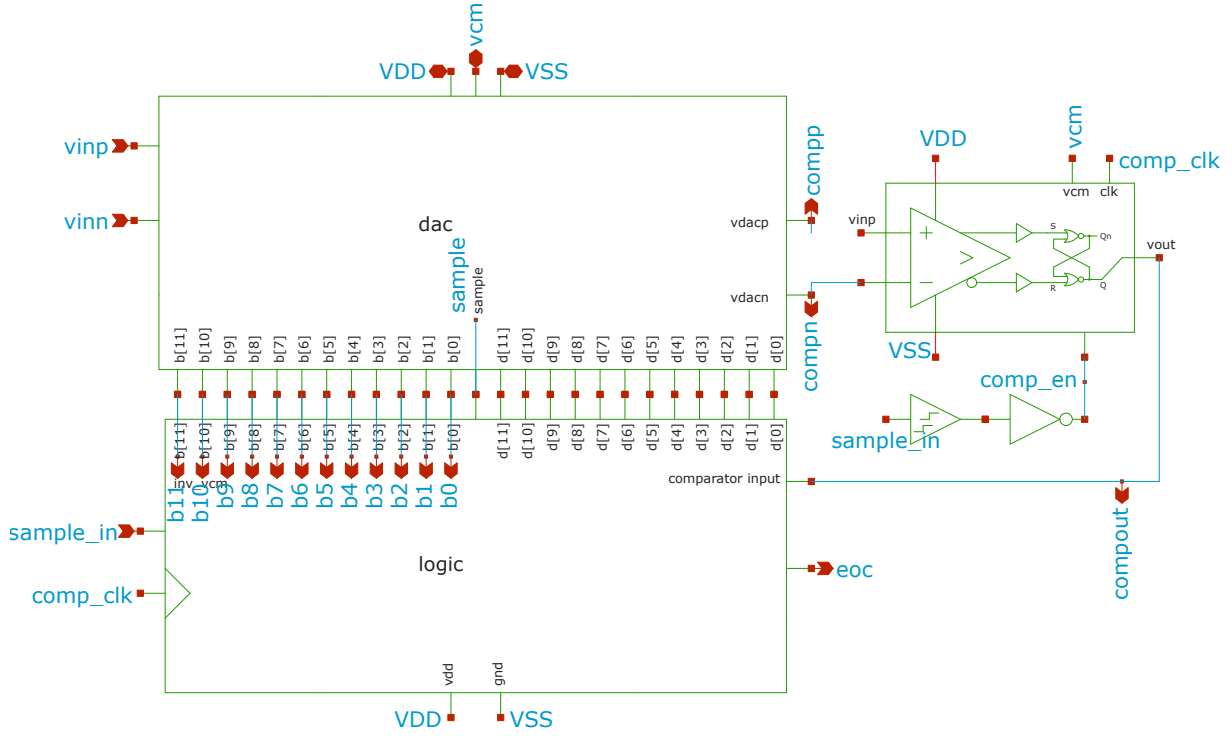


Figura 2.2: Diagrama de bloques en Xschem

## 2.1. Descripción del SAR ADC nivel TOP

Como se muestra en la la figura 2.1 las entradas del circuito son  $V_{inp}$ ,  $V_{inn}$  y  $sample$ , mientras que sus salidas son el vector  $D[11:0]$  y la señal de final de conversión  $eoc$  (end of conversion). El funcionamiento básico es el siguiente, mientras la señal  $sample$  esta en alto todas las salidas del ADC pasan a 0,  $eoc$  va a bajo y el ADC está siguiendo la entrada, cuando  $sample$  pasa a bajo, el ADC retiene la entrada y empieza a convertir bit a bit, hasta resolver el ultimo bit, cuando resuelve el ultimo bit,  $eoc$  se pone a alto indicando que el resultado esta listo, y el ADC queda esperano una nueva señal de sampleo. El hecho que la señal de  $sample$  sea externa está pensado para implementar un time-interleaving usando varios canales como este en paralelo coordinados por sus señales de sampleo individuales. Notar que el tiempo de sampleo ( $sample$  en 1) se fijó en 2ns (por elección de diseño) y el timing crítico estará en el flanco de bajada de  $sample$  (inicio de conversión) ya que la tension de entrada en este momento será la que se convierta. Un análisis detallado del sampleo en un tiempo no nulo se puede consultar en [6].

### 2.1.1. Arquitectura Inverted Merged Capacitor Switching (IMCS)

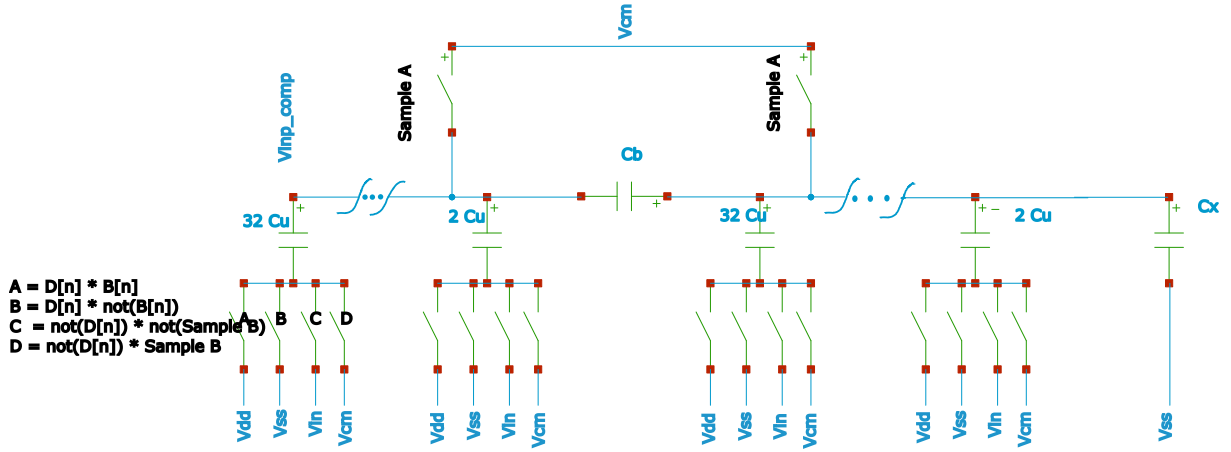


Figura 2.3: Diagrama simplificado de la arquitectura IMCS, usando split dac y capacitores de grounding

La arquitectura IMCS (Inverted Merged Capacitor Switching) es una técnica de conmutación para ADCs SAR que representa una evolución de la arquitectura MCS (Merged Capacitor Switching), propuesta por Chang [7]. Esta arquitectura fue seleccionada para el presente diseño por sus múltiples ventajas en términos de eficiencia energética, robustez y simplicidad de implementación. Se ejemplifica de manera simplificada la arquitectura implementada en la la figura 2.3

#### Beneficios de la Arquitectura IMCS

La arquitectura IMCS ofrece las siguientes ventajas respecto a otras técnicas de conmutación:

- **Inmunidad a Capacitancias Parásitas:** A diferencia de otras técnicas eficientes como la conmutación monotónica o la MCS estándar, la arquitectura IMCS garantiza que el voltaje en la placa superior (top-plate) del DAC comience y termine en el mismo voltaje ( $V_{CM}$ ). Esto hace que la precisión de la conversión sea insensible a las capacitancias parásitas en la placa superior del DAC y elimina la dependencia de la inyección de carga con la señal [7].
- **Alta Eficiencia Energética:** La IMCS consume un 94 % menos de energía en comparación con la implementación de conmutación convencional, igualando la eficiencia energética de la arquitectura MCS estándar.
- **Modo Común Constante:** Al igual que la MCS, la IMCS mantiene un voltaje de modo común constante durante el proceso de conversión. Esto simplifica el diseño del comparador y mejora la linealidad del ADC al evitar variaciones en el offset dinámico del comparador.
- **Rango de Entrada Rail-to-Rail:** Permite un rango de entrada completo (rail-to-rail), lo que maximiza la relación señal-ruido (SNR).

La Tabla la tabla 2.1 resume las características principales de las diferentes arquitecturas de conmutación.

Cuadro 2.1: Comparación de arquitecturas de conmutación CDAC

| Característica                   | Conv.     | Monotonic     | MCS           | IMCS          |
|----------------------------------|-----------|---------------|---------------|---------------|
| Consumo relativo                 | 100 %     | 19 %          | 6 %           | 6 %           |
| Sensibilidad parásitos top-plate | Media     | Alta          | Alta          | Baja          |
| $V_{CM}$ constante               | No        | No            | Sí            | Sí            |
| Rango entrada                    | Limitado  | Limitado      | Rail-to-rail  | Rail-to-rail  |
| Capacitancia total               | $2^N C_u$ | $2^{N-1} C_u$ | $2^{N-1} C_u$ | $2^{N-1} C_u$ |

### Funcionamiento del Esquema IMCS

El nombre “Inverted” (Invertido) proviene de la forma en que se realiza el muestreo en comparación con la MCS estándar. El proceso de conversión se desarrolla en las siguientes fases:

1. **Fase de Muestreo:** En lugar de muestrear la señal de entrada en la placa superior, la arquitectura IMCS muestrea el voltaje de modo común ( $V_{CM}$ ) en las placas superiores del array de capacitores y la señal de entrada diferencial ( $V_{IN}$ ) en las placas inferiores. Para una implementación diferencial, las placas inferiores del array positivo se conectan a  $V_{INP}$  y las del array negativo a  $V_{INN}$ .
2. **Fase de Inversión:** Una vez completado el muestreo, se desconectan las fuentes de entrada mediante la apertura de los switches controlados por las señales non-overlap. Luego, se conecta el voltaje  $V_{CM}$  a las placas inferiores de todo el array. Esto empuja la diferencia de voltaje (proporcional a  $-V_{IN}$ ) hacia las placas superiores, donde se encuentra conectado el comparador. El primer bit (MSB) se determina inmediatamente sin consumir energía del array de capacitores.
3. **Ciclos de Conversión (Bit Cycling):** Dependiendo de la decisión del comparador, los capacitores correspondientes se conmutan desde  $V_{CM}$  hacia la referencia positiva ( $V_{DD}$ ) o negativa ( $V_{SS}$ ). El objetivo del algoritmo es llevar el voltaje de la placa superior de regreso a  $V_{CM}$  paso a paso, reduciendo el error en cada ciclo mediante búsqueda binaria. Este proceso se repite para cada uno de los 12 bits de resolución.

En resumen, la IMCS combina la eficiencia energética de la MCS con la robustez frente a parásitos de las arquitecturas de muestreo en placa inferior (bottom-plate sampling). Aunque la arquitectura IMCS también permite implementar redundancia con ventanas de tolerancia simétricas, esta característica no fue implementada en el presente diseño, optando por una configuración binaria pura de 12 bits.

La característica distintiva que hace a la IMCS insensible a parásitos es que los voltajes en las placas superiores comienzan en  $V_{CM}$  y convergen hacia  $V_{CM}$  al final de la conversión, eliminando el efecto de las capacitancias parásitas en la precisión de la conversión.

## 2.2. DAC Capacitivo (CDAC)

El CDAC implementa el esquema de conmutación IMCS (Inverted Merged Capacitor Switching) con una arquitectura split-binary de 6 bits MSB + 6 bits LSB sin redundancia, para un total de 12 bits de resolución. Esta arquitectura incluye por cada capacitor 4 switches (Una Transmission Gate para conectar  $V_{in}$ , otra para conectar  $V_{cm}$ , un PMOS para conectar a VDD y uno NMOS para conectar a aVSS) en la bottom plate, y 4 swtiches generales que manejan las Top plates del MSB y LSB del lado positivo como del negativo para el sampleo. Además se agregaron capacitores de grounding para solucionar prblemas de implementación del capacitor de puente (C bridge,  $C_B$ ) y de overrange en el Sub-DAC[7].

### 2.2.1. Definición del Capacitor Unitario ( $C_u$ )

#### Criterio de Ruido Térmico

El tamaño del capacitor unitario se determina principalmente por el compromiso entre ruido térmico ( $kT/C$ ) y área de silicio. Para un ADC de 12 bits, el ruido térmico total debe ser suficientemente pequeño para no degradar el número efectivo de bits (ENOB).

Donde  $k$  es la constante de Boltzmann,  $T$  la temperatura absoluta,  $V_{ref}$  el voltaje de referencia y  $N$  la resolución en bits. Como se indica en la figura, el número efectivo de bits (Effective Number of Bits, ENOB) decrece con la magnitud del ruido térmico. En nuestro caso, buscaremos perder menos de 1 bit de ENOB, por lo que buscaremos un ruido térmico menor a 0,5 LSB.

El criterio utilizado establece que la capacitancia total del array debe satisfacer, usamos  $T=300K$ :

$$C_{total} > \frac{kT \cdot 2^{N+1}}{V_{ref}^2} \quad (2.1)$$

$$C_{total} > 85,7 fF \quad (2.2)$$

Esto refiere a la capacidad total del CDAC. Para obtener el valor unitario debemos dividirlo en la cantidad de capacitores unitarios "vistos" por el switch de muestreo y retención (sample & hold, S&H). Esto se aclara en la sección 2-1-3, en este caso son:

$$C_{total} = C_{MSB} + C_{LSB} \quad (2.3)$$

$$C_{MSB} = 63C_u, C_{LSB} = 63C_u + C_x = 63C_u + 31C_u, \quad (2.4)$$

$$C_{total} = 63C_u + (63C_u + 31C_u) = 157C_u \quad (2.5)$$

Finalmente:

$$C_u \geq C_{totalmin}/157 \quad (2.6)$$

$$C_u \geq 0,54 fF \quad (2.7)$$

### 2.2.2. Implementación del Array Split-Binary

Para El array se divide en:

- **MDAC (Main DAC):** 6 bits MSB con capacitores pesados binariamente:  $32C_u, 16C_u, 8C_u, 4C_u, 2C_u, 1C_u$

- **Capacitor de puente:**  $C_B = 2C_u$
- **SDAC (Sub DAC):** 6 bits LSB con capacitores pesados binariamente:  $32C_u, 16C_u, 8C_u, 4C_u, 2C_u, 1C_u$
- **Capacitor de grounding:**  $C_X = 31C_u$  en paralelo con el SDAC

### Selección del capacitor unitario $C_u$

Se utilizaron capacitores MIM (Metal-Insulator-Metal) del PDK ihp-sg13g2 por simplicidad en el diseño del layout.

- En la documentacion no se indica el coeficiente de matching
- Bajo coeficiente de tensión y temperatura
- Estructura planar que facilita el layout centroe común (common-centroid)
- Se partió del tamaño mínimo permitido por el proceso para un mimcap,  $W_c = L_c = 1,15\mu m$  con  $C = 2,16fF$
- A través de iteraciones sobre la simulación TOP se encontró que era necesario un capacitor mucho más grande para lograr una conversión correcta. Algunos efectos que son mitigados por aumentar  $C_u$  son: Inyección de carga, corrientes de fuga, ruido por Kickback del comparador. Mediante simulaciones iterativas a nivel de sistema, se observó que el valor mínimo teórico era insuficiente debido a efectos de segundo orden como la inyección de carga y el kickback noise del comparador. Se seleccionó finalmente un capacitor unitario  $W_c = L_c = 3\mu m$  con  $C = 13,9fF$ , lo cual provee un margen de robustez adicional.

El capacitor unitario implementado tiene un valor de  $C_u = 15 \text{ fF}$  con dimensiones de  $3 \mu m \times 3 \mu m$ .

### 2.2.3. Cálculo de Capacitores de Atenuación ( $C_B$ ) y Grounding ( $C_X$ )

#### Problema del Capacitor Fraccional

En arquitecturas split-binary, el capacitor de puente ( $C_B$ ) que conecta los segmentos MSB y LSB tradicionalmente tiene un valor fraccional:

$$C_B = \frac{2^L - 1}{2^L} C_u \quad (2.8)$$

Para  $L = 6$  bits en el segmento LSB, esto resulta en  $C_B = 63/64 \cdot C_u$ , un valor no entero que dificulta el matching en el layout.

#### Problema de los rieles del Sub-DAC

Los rieles del sub-dac pueden salir de los rieles de alimentación, como se explica en [7]. Esto puede producir que las juntas de drenador de los switches se polaricen en directa, por ejemplo, si el drenador de un pmos (con sustrato a vdd) se va arriba de VDD por encima de la tensión de umbral de esta junta  $V_{thp} = 0,48V$  posiblemente dañado el chip.

## Metodología de Cálculo

Para resolver ambos problemas, se implementó la arquitectura propuesta por Chang [7], que introduce un capacitor de grounding ( $C_X$ ) permitiendo que  $C_{BX}$  sea un valor entero de unidades  $C_u$ .

Se adjunta un archivo de Maxima (disponible en el repositorio GitHub: [https://github.com/gsuckz/SAR\\_ADC\\_IHP](https://github.com/gsuckz/SAR_ADC_IHP)) que calcula el valor de  $C_X$  (como múltiplo de  $C_u$ ).

## Valores Finales

Los valores calculados son:

- $C_B = 2 \cdot C_u = 27,8fF$
- $C_X = 31 \cdot C_u = 430,9fF$

Estos valores permiten:

- Fabricación precisa con matching controlado (valores enteros)
- Prevención de over-range en el nodo LSB durante conmutaciones

Sin embargo, los pasos del LSB estaban atenuados, por lo que se iteró sobre el  $W_B$  al aumentar el valor de  $C_B$  se puede disminuir la atenuación e incluso hacer los pasos mas grandes de lo necesario, se encontro que el valor  $W_B = 3,23\mu m$  entrega los pasos correctos. Se procedió como sigue:

- Se calcula la atenuación A del paso como:  
$$A = \frac{V_{PasoReal}}{V_{PasoIdeal}}$$
- Se divide  $W_B$  por la atenuación obtenida, tal que:  
$$W_{Bnuevo} = \frac{W_B}{A}$$
- Volver a calcular A
- Repetir hasta encontrar un valor aceptable ( $A \approx 1$ )

### 2.2.4. Logica de las señales de control de la Bottom Plate de cada celda

Cada capacitor unitario del array implementa el esquema IMCS mediante la siguiente lógica de switches:

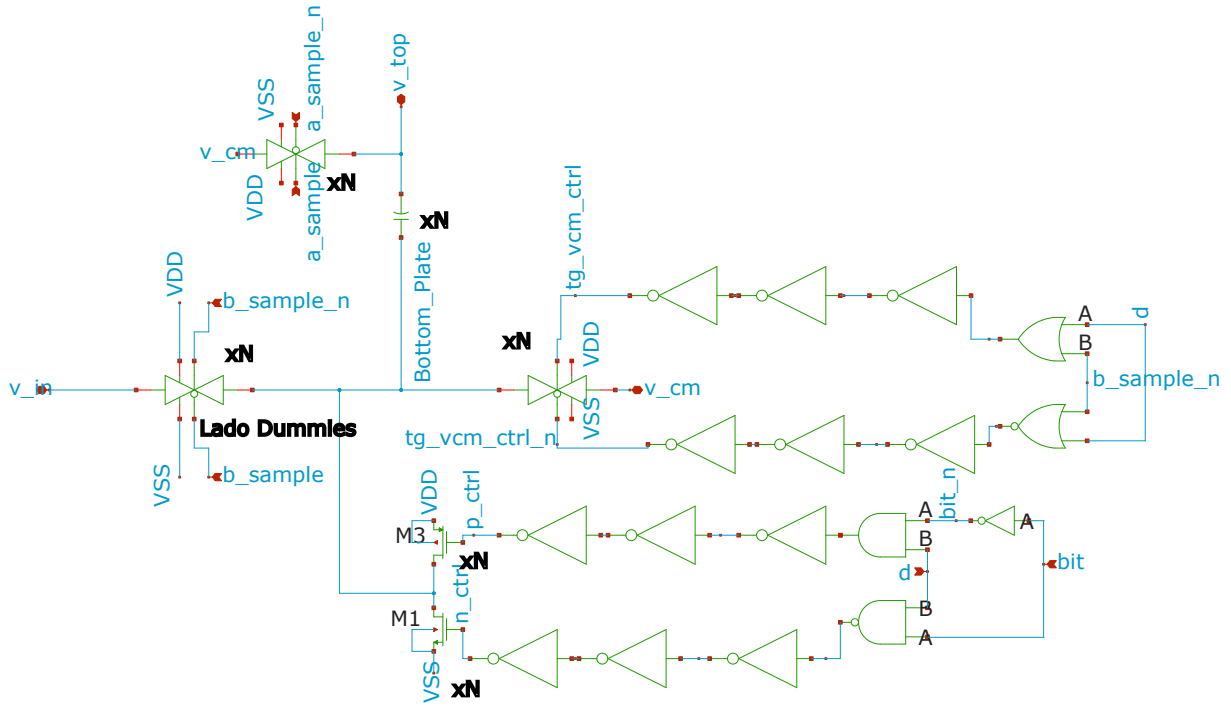


Figura 2.4: Esquema de la lógica de la celda unitaria del DAC, notar que se indican los dispositivos escalados por unidad de capacitor. La cadena de buffers solo es ilustrativa.

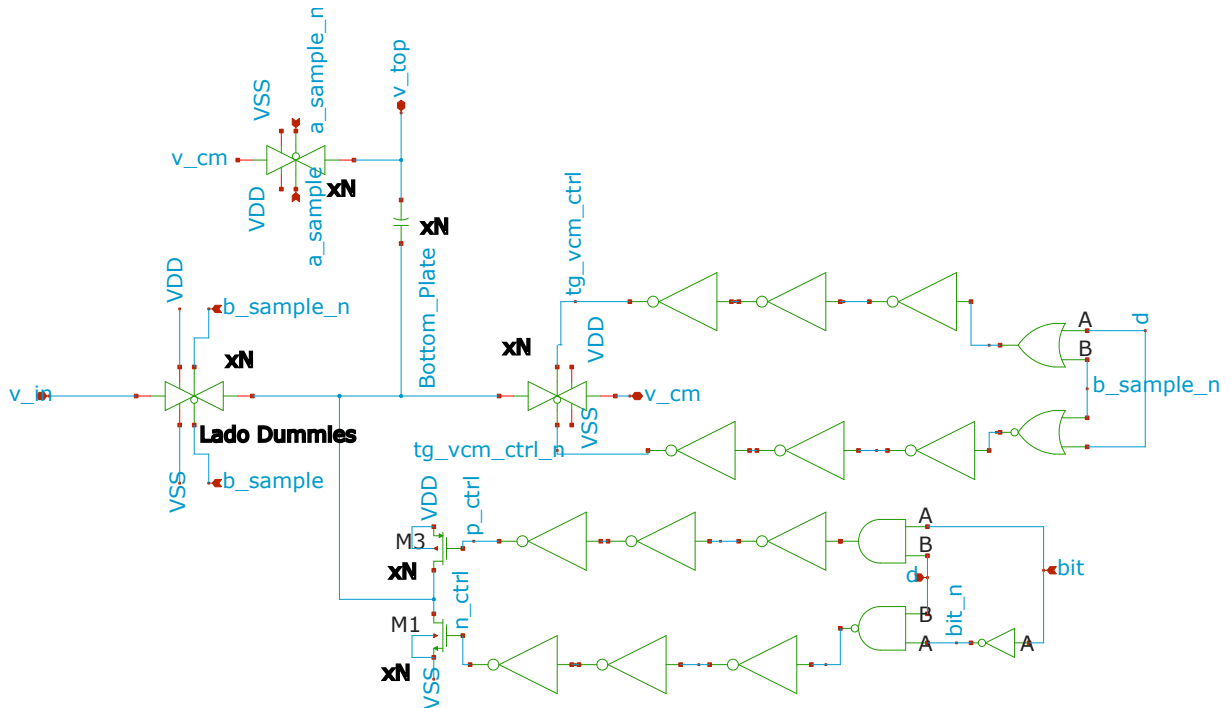


Figura 2.5: Celda con bit negado. La cadena de buffers solo es ilustrativa.

### Switches de Placa Inferior (Bottom Plate)

La lógica para activar los switches segun las señales  $B[n]$  y  $D[n]$  se detallan en la figura 2.4 y la figura 2.5

- **Conexión a entradas ( $V_{INP}$ ,  $V_{INN}$ ):** Transmission gates con dummies para minimizar charge injection durante el muestreo
- **Conexión a modo común ( $V_{CM}$ ):** Transmission gates sin dummies (switching durante conversión)
- **Conexión a  $V_{DD}$  (referencia):** Transistor PMOS individual
- **Conexión a  $V_{SS}$  (tierra):** Transistor NMOS individual

Los transistores PMOS y NMOS tienen el mismo tamaño que los transistores que componen los transmission gates para mantener simetría en las impedancias de conmutación.

### 2.2.5. Generador de Señales Non-Overlap

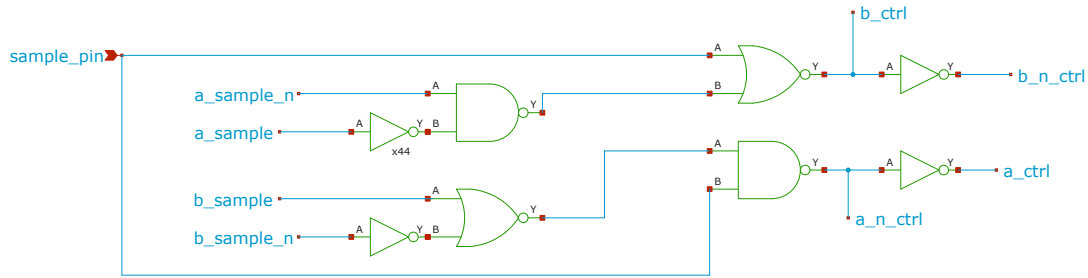


Figura 2.6: Diagrama del generador de señales de sampleo sin superposición

#### Funcionamiento

El generador de pulsos de la figura 2.6 produce dos señales para el sampleo ( $Sample_A$  y  $Sample_B$ ), la primera es la que conecta las Top Plates a  $V_{cm}$ , mientras que la segunda conecta las Bottom Plates a  $V_{in}$ . Funciona de la siguiente manera:

- $Sample_A$  y  $Sample_B$  cierran sus switches cargando los capacitores del CDAC
- $Sample_A$  Abre, dejando flotantes las TOP Plates
- $Sample_B$  Abre, desconectando las Bottom Plates de  $V_{in}$ , cuando esto sucede, la lógica correspondiente a esa celda del CDAC, conecta la Bottom plate a  $V_{cm}$  de acuerdo a la arquitectura IMCS. Permanecerá conectada, hasta que se decida el bit correspondiente a la celda.

Este procedimiento evita que los capacitores pierdan carga si se conectaran ambas placas al mismo potencial  $V_{cm}$ . Para asegurar el NOL se realizó esta implementación a lazo cerrado, por lo que el tiempo muerto incluye todos los delays de los buffers. Las señales se muestran en la figura 2.7, se indica el delay en la figura 2.8

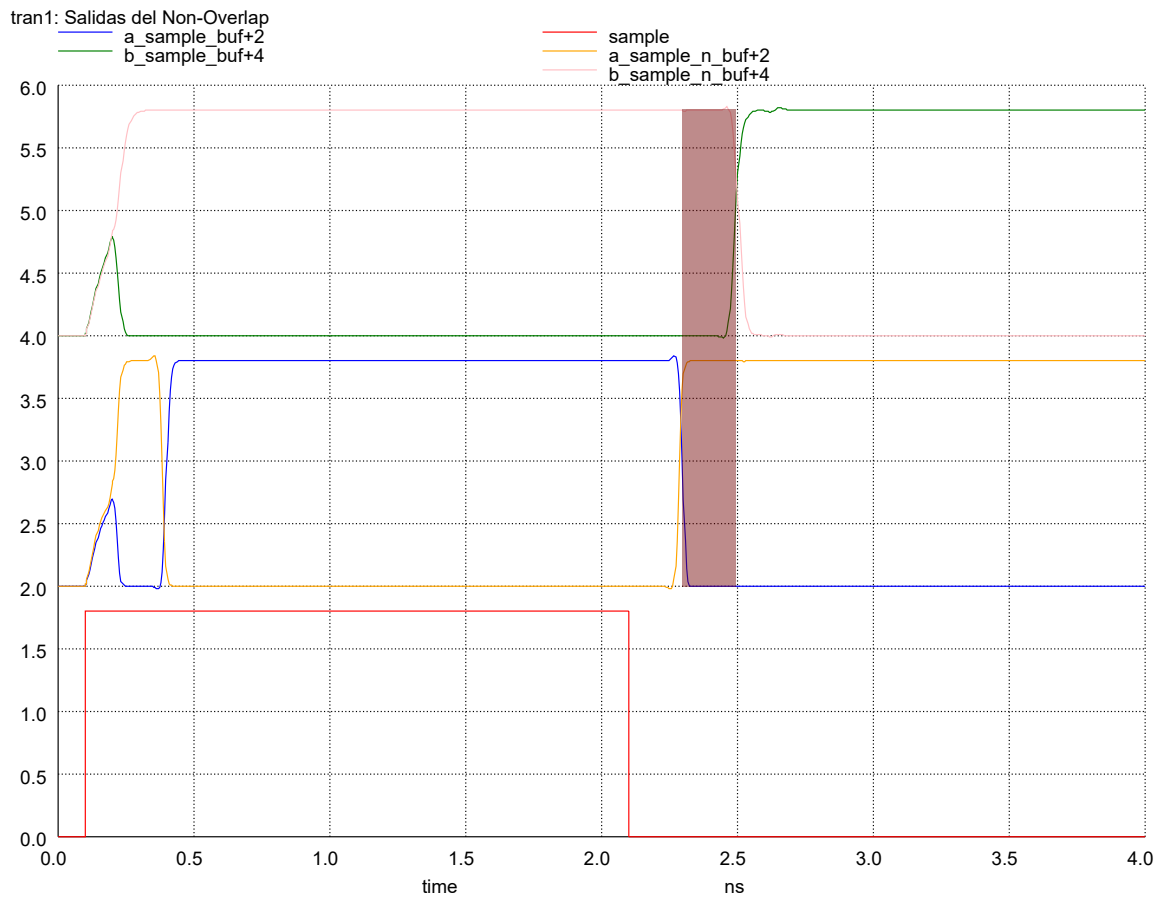


Figura 2.7: Salidas del NOL, se resalta el tiempo muestro donde ambas señales abren los switches

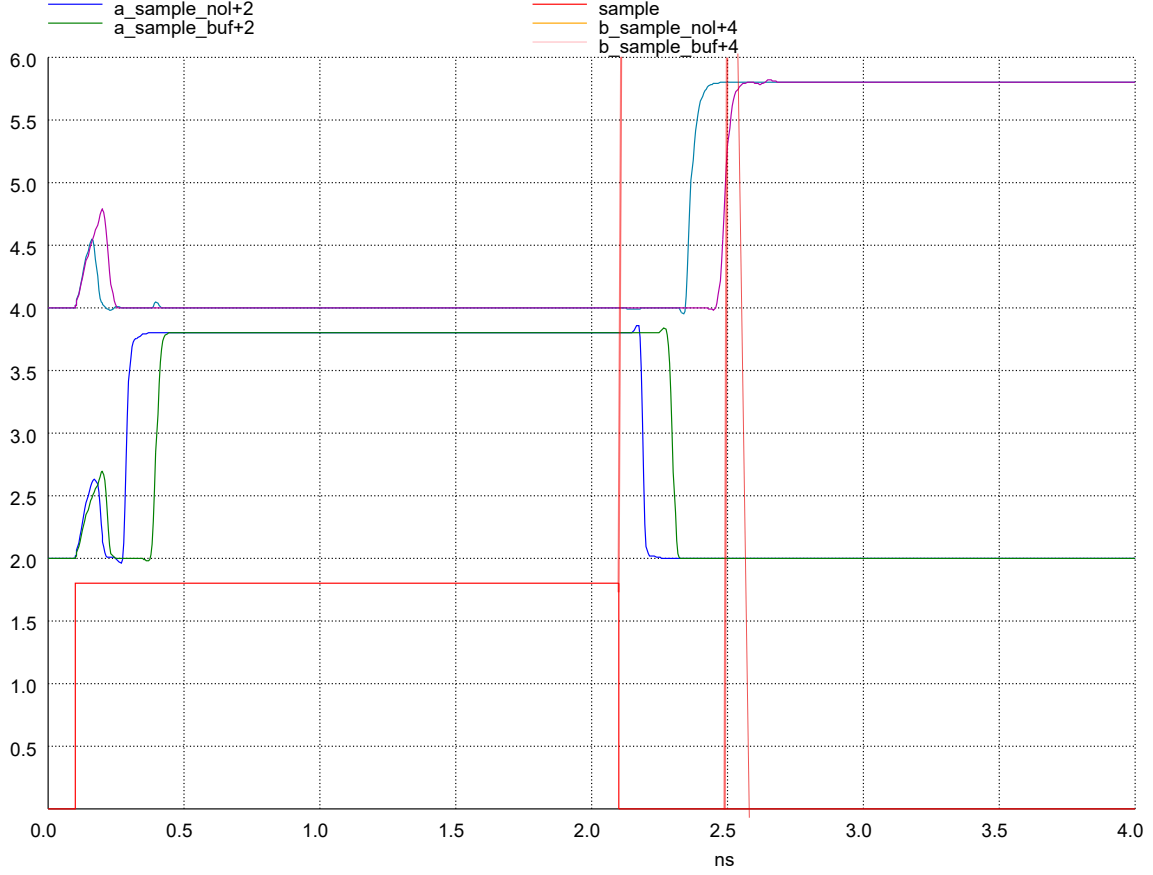


Figura 2.8: Señales generadas por el NOL, se señala el alto delay generado por la operacion a lazo cerrado

### 2.2.6. Top Plate Sample switch

Se implementaron 4 compuertas de transmisión, 2 para los nodos de las TOP plates MSB y LSB de cada parte del DAC

### 2.2.7. Requerimiento de Settling

#### Presupuesto de Tiempo

Para una velocidad de muestreo objetivo de 40 MSa/s (período de 25 ns), adoptamos un tiempo de muestreo (Tiempo de sampling)  $T_s = 2ns$ , restándonos 23ns para realizar la conversión. Por lo que nuestra señal se debe estabilizar en menos de ese tiempo (tiempo de settling en la respuesta al escalón). Además buscaremos que el ancho de banda 3dB (BW) del S&H sea al menos 20 veces mayor a la frecuencia de Nyquist de este ADC, en este caso buscamos un  $BW = 400MHz$ . En la figura 2.13 se muestra el resultado al samplear una señal con  $f_1 = 4 * f_{nyquist}$

Usaremos la regla practica de [10]:

$$BW = \frac{0,35}{T_r}$$

$$T_r = \frac{0,35}{BW}$$

$$T_r = \frac{0,35s}{400 * 10^6} = 875ps$$

### 2.2.8. Dimensionamiento del Transmission Gate (sample)

Se realizó un test bench con el objetivo de alcanzar una respuesta al escalon con un tiempo de settling menor a 875ps. Para el mismo se colocó una compuerta de transmission con dummies, el capacitor y una compuerta de transmission sin dummies, como decision de diseño, las dimensiones de  $W_n$  y  $W_p$  serán las mismas para todas las compuertas.

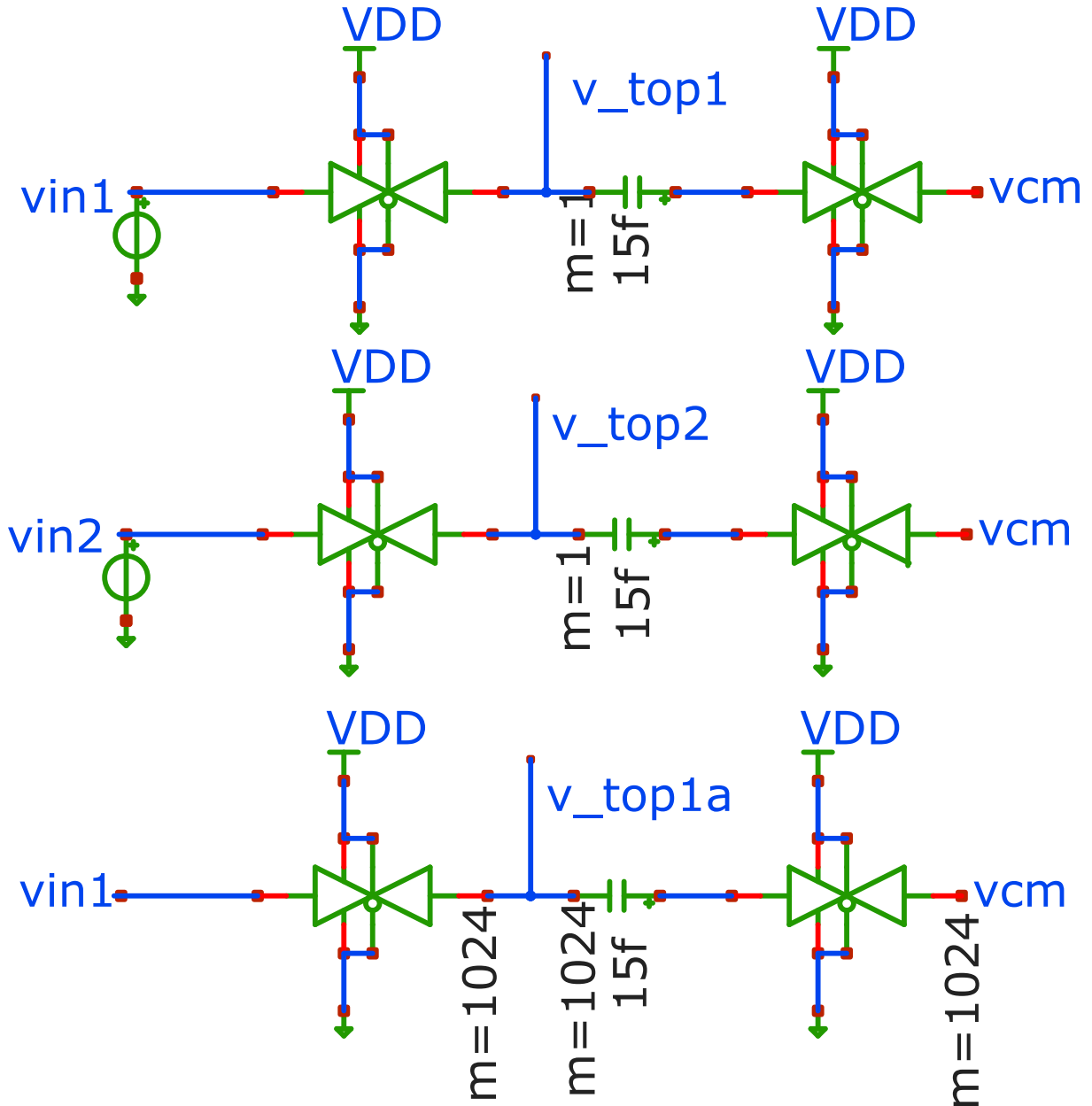


Figura 2.9: Testbench usado para probar el  $T_r$  del S&H

Se Realizaron las siguientes pruebas:

- Se busco un tiempo de settling menor a 875ps para test bench de la figura

- Se comprobó el efecto de agregar dummies para mitigar la inyección de carga, además se probó la dependencia de este método respecto a tiempo de bajada de la señal de sample. la figura 2.15 la figura 2.16
- Se comprobó que si ponemos n compuertas en paralelo manejan n capacitores igual, el tiempo de rise se mantiene. la figura 2.11
- Se probó la respuesta al escalon en la señal de sample (tiempo que tarda el switch en cerrar) y luego se probó la respuesta a los 2 escalones en simultaneo. la figura 2.12

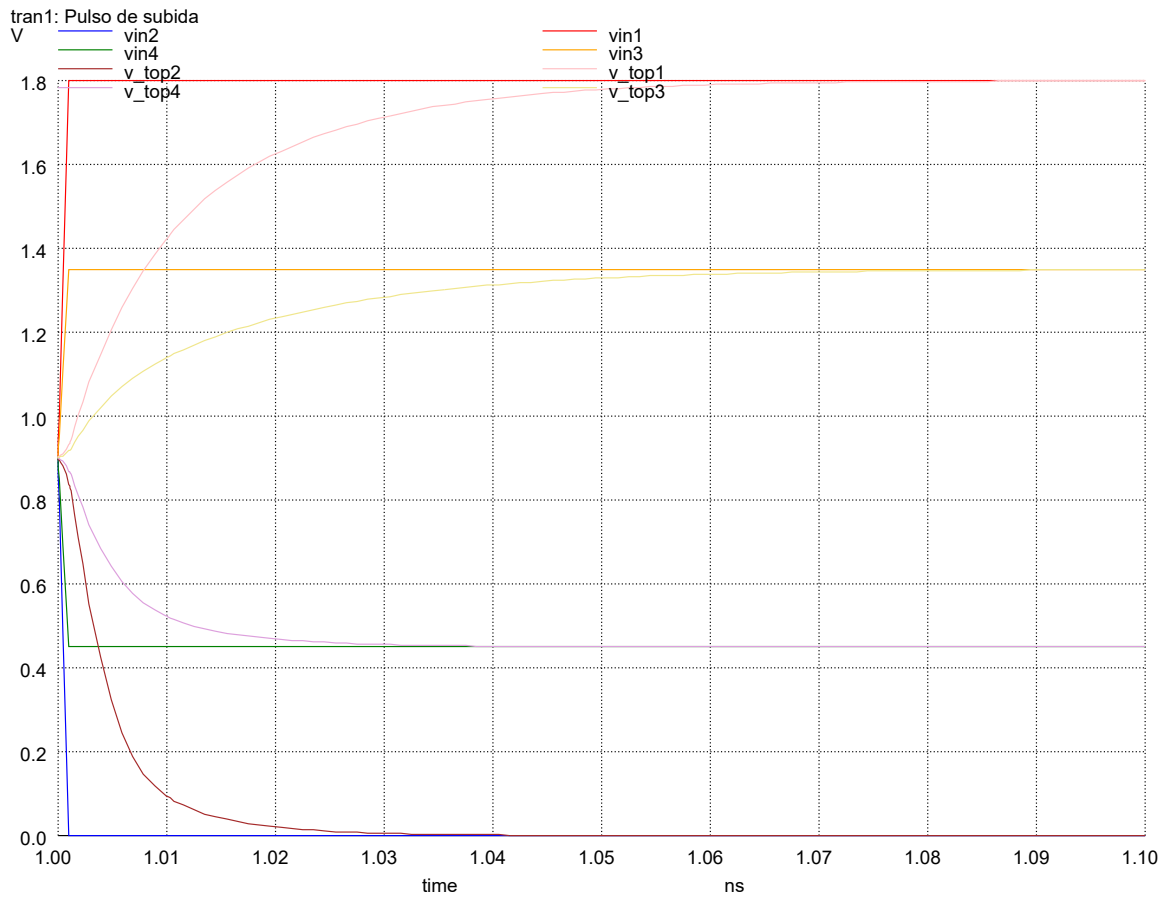


Figura 2.10: Respuesta al escalón (subida)

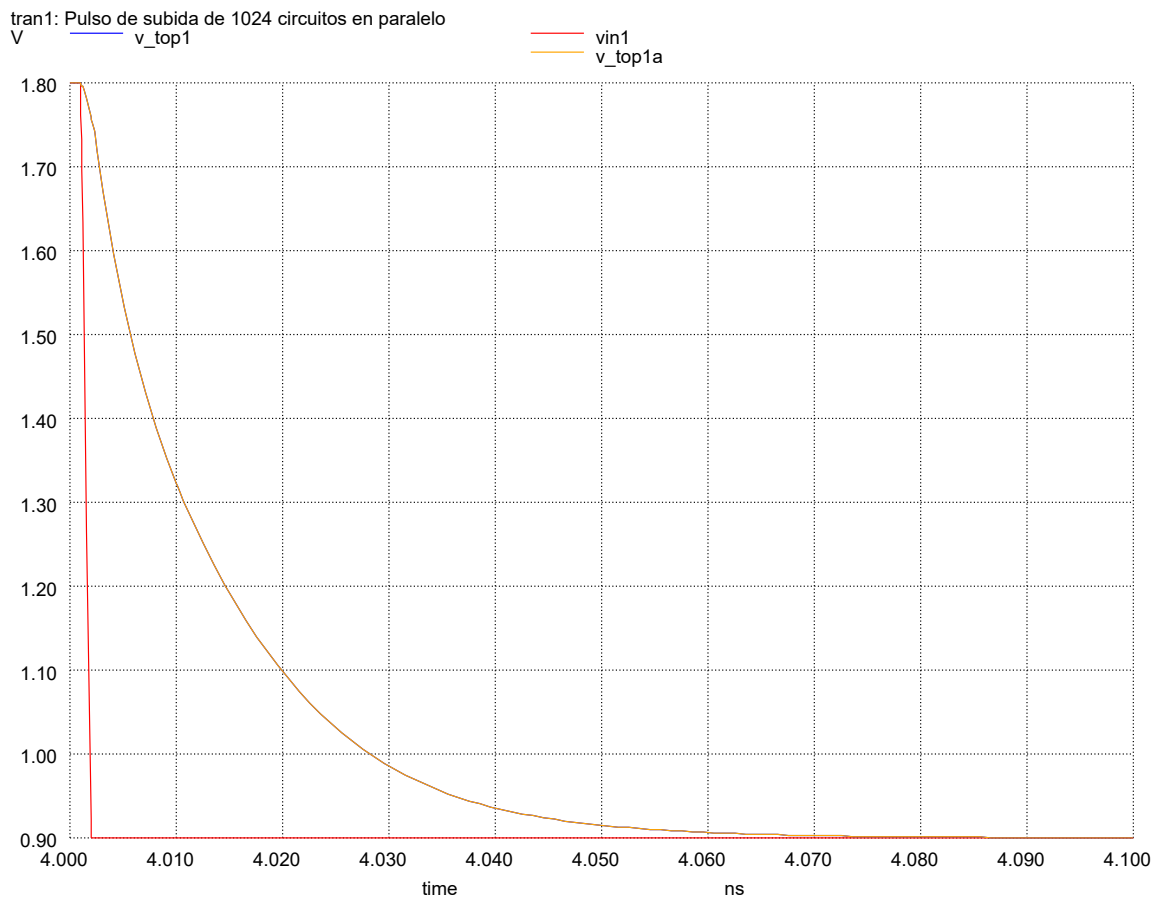


Figura 2.11: Respuesta al escalón de varios en paralelo (v\_top1a) se observa que se superpone con la respuesta de 1 solo conjunto (v\_top1)

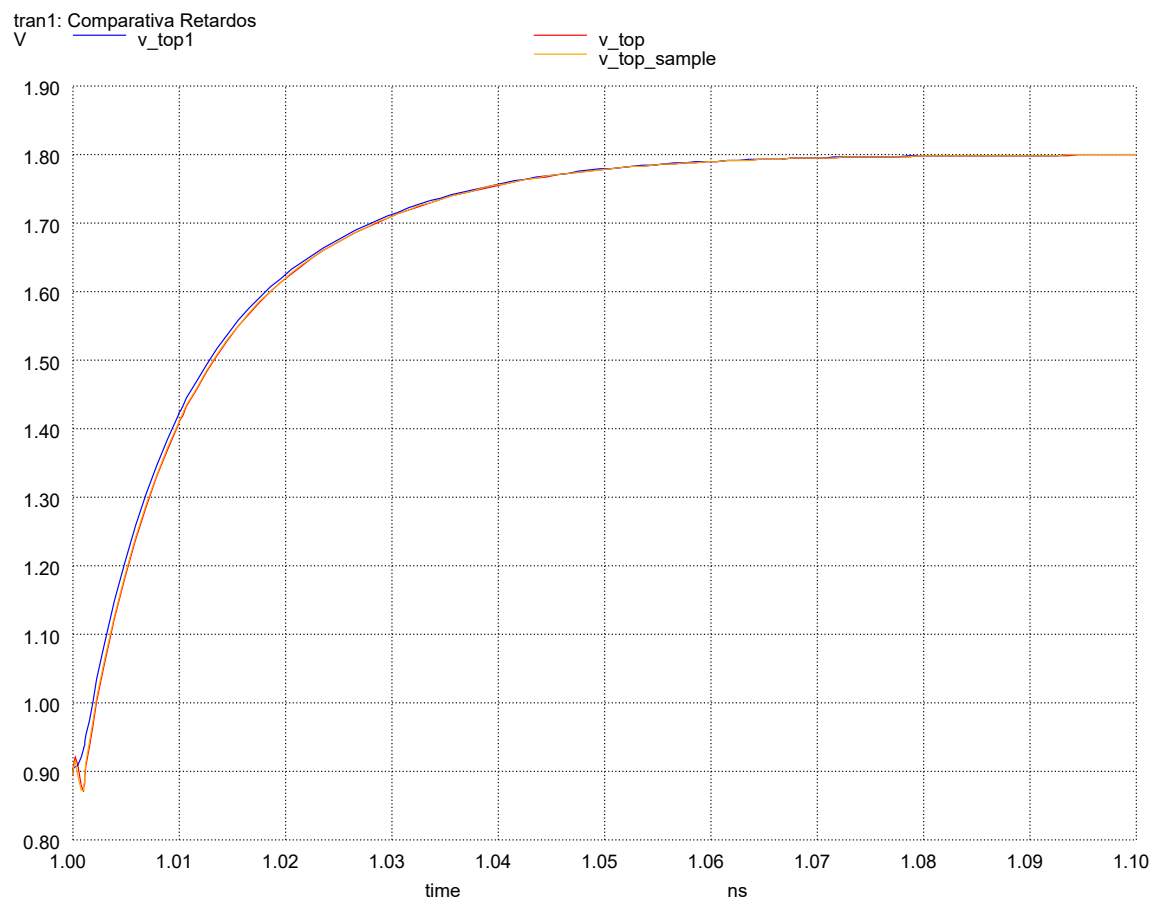


Figura 2.12: Respuesta del circuito con escalon en: entrada (v\_top1), señal de sampleo(v\_top\_sample) y en ambos (v\_top)

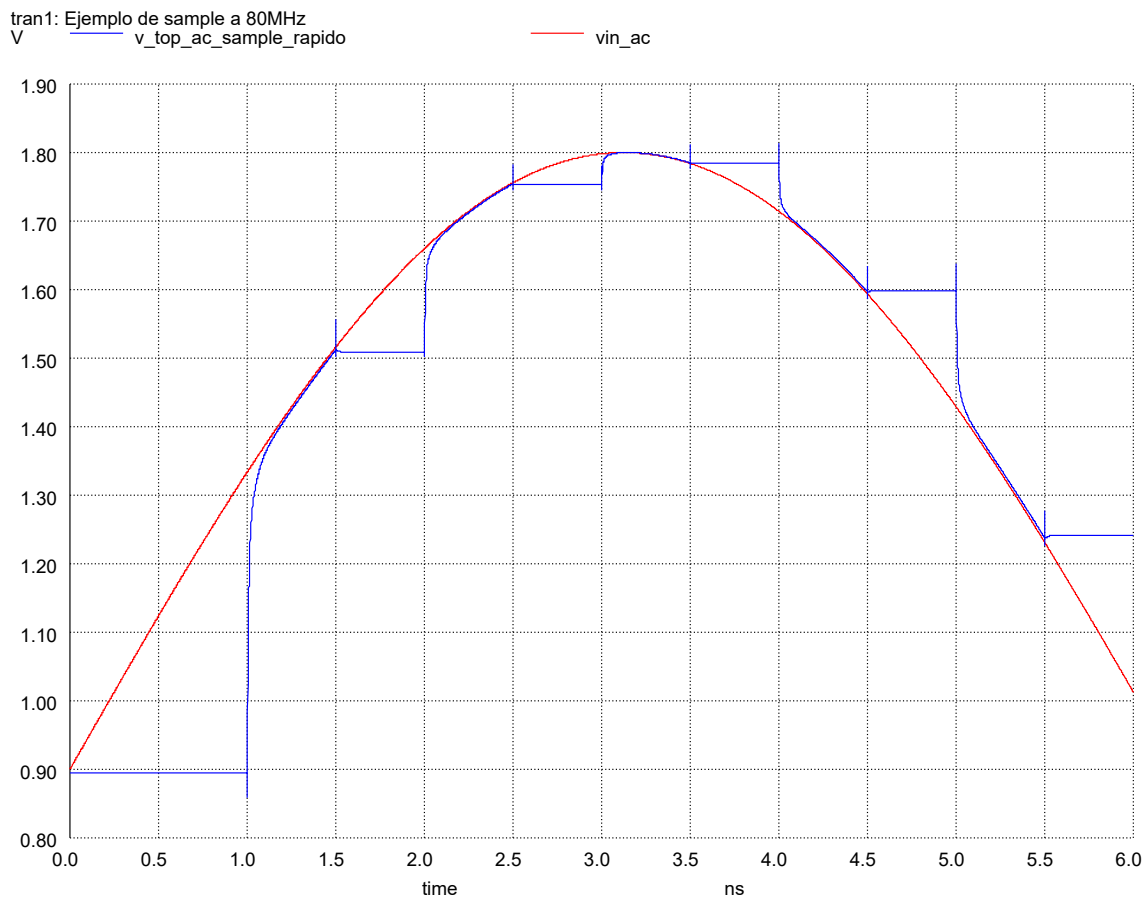


Figura 2.13: Ejemplo de sampleo de una señal de 80MHz (4 veces la frecuencia de Nyquist)

Cuadro 2.2: Dimensiones del Transmission Gate

| Parámetro                   | Valor                                  |
|-----------------------------|----------------------------------------|
| $W_n / L_n$                 | 1.0 $\mu\text{m}$ / 0.13 $\mu\text{m}$ |
| $W_p / L_p$                 | 2.0 $\mu\text{m}$ / 0.13 $\mu\text{m}$ |
| Tiempo de settling objetivo | < 875 ps                               |

### 2.2.9. Compensación de Inyección de Carga con Dummies

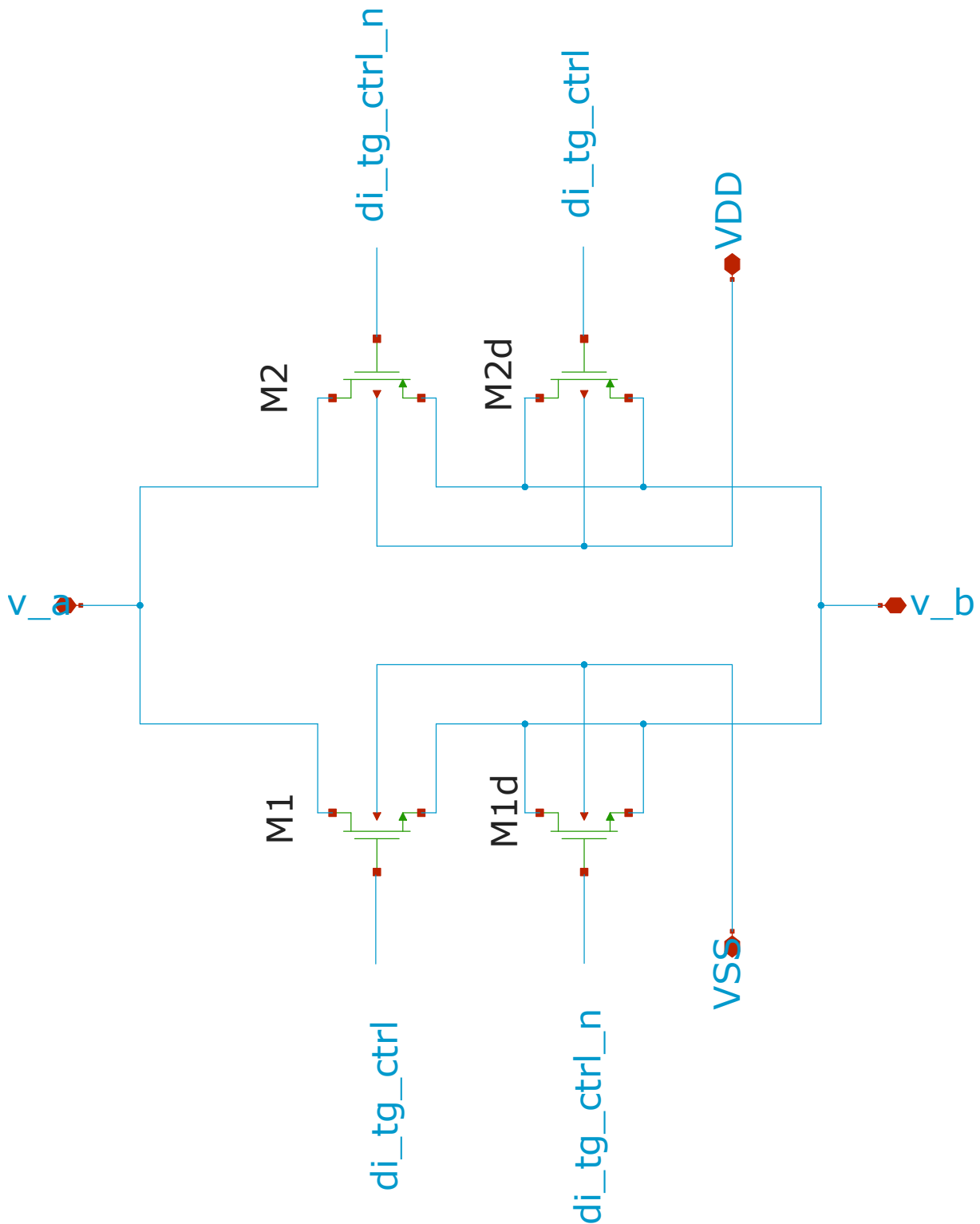


Figura 2.14: Transmission gate implementada usando dummies para mitigar la inyección de carga

## Problema de Charge Injection

Cuando los transmission gates se abren, la carga almacenada en el canal de conducción se inyecta parcialmente en el capacitor de muestreo, introduciendo un error dependiente de la señal.

## Implementación de Dummies

Se implementaron transistores dummy conectados con compuerta controlada por la señal complementaria. Partiendo de que la capacidad de canal se descarga a través de Source y Drain por igual, por lo que un transistor igual, con la mitad del area (en este caso w) absorberá la mitad de la carga que iría de otra forma al capacitor. Las la figura 2.15 y la figura 2.16 y muestran que para que esta compensacion sea efectiva necesitamos un tiempo de bajada corto.

$$W_{dummy} = 0,5 \times W_{active} \quad (2.9)$$

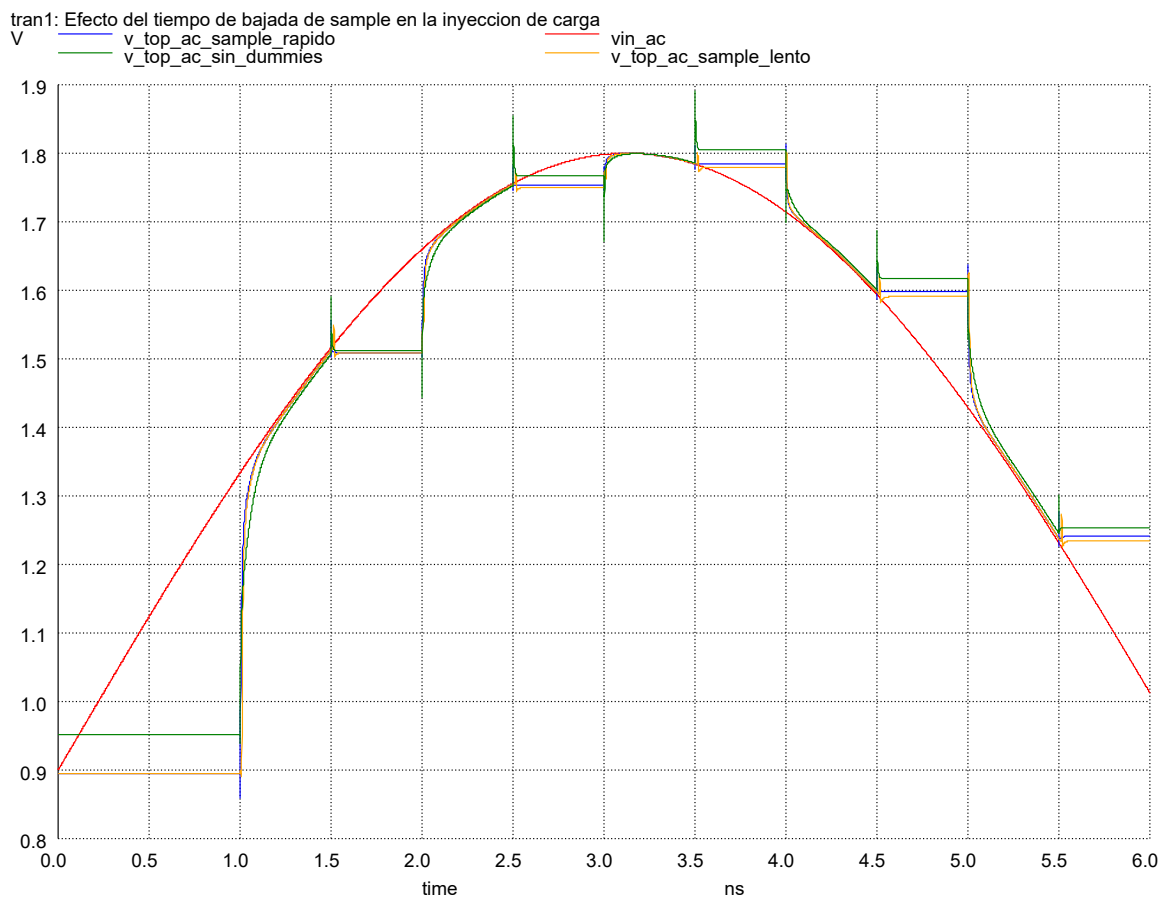


Figura 2.15: Dependencia de la compensación con dummies del falltime de la señal de sample

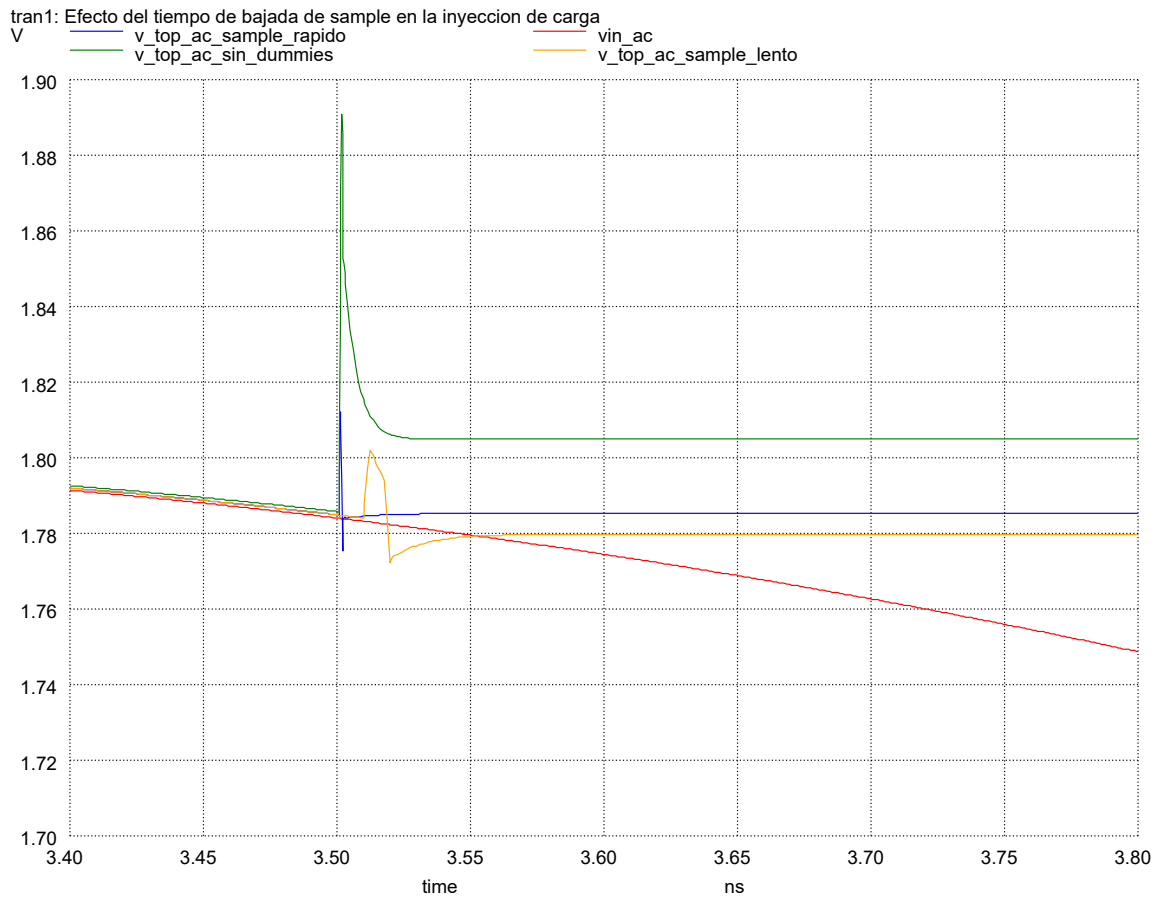


Figura 2.16: Dependencia de la compensación con dummies del falltime de la señal de sample (zoom)

## 2.2.10. Testbench del CDAC

### Objetivo del Test

El testbench valida:

1. Tiempo de settling de cada paso de conversión
2. Valores correctos de los saltos de tensión (verificación de atenuación)
3. Funcionamiento correcto de ambos sub-arrays (MDAC y SDAC)
4. Comportamiento del esquema de conmutación IMCS

### Configuración del Test

El testbench implementa:

- Generadores de voltaje para  $V_{INP}$ ,  $V_{INN}$ ,  $V_{CM}$ ,  $V_{DD}$
- Bloque lógica SAR para obtener el delay total desde la decisión del comparador
- Medición diferencial en las placas superiores del CDAC
- Análisis transitorio para observar settling

## Resultados

Cuadro 2.3: Valores de salto de tensión en el CDAC - Verificación del esquema IMCS

| Salto | Segmento | $V_{\text{dacP}}$ (mV) | $V_{\text{dacN}}$ (mV) | $V_{\text{dac,diff}}$ (mV) |
|-------|----------|------------------------|------------------------|----------------------------|
| 0     | Main DAC | 401,017                | 401,014                | 802,439                    |
| 1     | Main DAC | 200,648                | 200,649                | 401,297                    |
| 2     | Main DAC | 100,351                | 100,285                | 200,636                    |
| 3     | Main DAC | 50,188                 | 50,142                 | 100,331                    |
| 4     | Main DAC | 25,097                 | 25,064                 | 50,161                     |
| 5     | Main DAC | 12,549                 | 12,53                  | 25,079                     |
| 6     | Sub DAC  | 6,359                  | 6,368                  | 12,72                      |
| 7     | Sub DAC  | 3,185                  | 3,167                  | 6,352                      |
| 8     | Sub DAC  | 1,592                  | 1,582                  | 3,174                      |
| 9     | Sub DAC  | 0,79                   | 0,79                   | 1,59                       |
| 10    | Sub DAC  | 0,397                  | 0,397                  | 0,794                      |

Se implementaron las compouertas con las medidas detalladas en la tabla 2.2. Se puede observar en los cuadros 2.3 y 2.4 que los pasos decrecen exponencialmente, y que están atenuados respecto a los esperados por un factor de 0,89. Se observa además que la operación en ambos CDAC (lado p y lado n) son prácticamente simétricos.

Cuadro 2.4: Parámetros del CDAC

| Parámetro                              | Valor                                  |
|----------------------------------------|----------------------------------------|
| Capacitor unitario ( $C_u$ )           | 13.9 fF                                |
| Tipo de capacitor                      | MIM $3\mu\text{m} \times 3\mu\text{m}$ |
| Capacitor de puente ( $C_B$ )          | $2C_u = 27.8 \text{ fF}$               |
| Capacitor de grounding ( $C_X$ )       | $31C_u = 430 \text{ fF}$               |
| Capacitancia total MDAC                | $63C_u = 945 \text{ fF}$               |
| Capacitancia total SDAC + $C_X$        | $94C_u = 1.3 \text{ pF}$               |
| Tiempo de settling promedio (por paso) | 387 ps                                 |
| Tiempo de settling Máximo (por paso)   | 527 ps                                 |

## 2.3. Comparador de Doble Cola

Se implementó el comparador de doble cola (double-tail) propuesto por Bindra et al. [5], que separa las etapas de preamplificación y regeneración para optimizar velocidad y consumo.

El comparador de doble cola (Fig. 2.2) Se compone de 2 etapas, un preamplificador que aporta ganancia a la vez que aislación hacia la etapa anterior (CDAC) y una segunda etapa con un latch regenerativo para realizar la comparación.

El funcionamiento básico se describe así: cuando  $di\_clk$  está en 0, el preamp carga los nodos  $vcpp$  y  $vcpn$  a VDD. Mientras que el latch descarga sus nodos de salida hacia VSS. Este tiempo se llama tiempo de reset. Cuando  $di\_clk$  está en 1, el preamp empieza a descargar los nodos  $vcpp$  y  $vcpn$  a través del capacitor de cola. Esto genera una tensión diferencial en la entrada del latch regenerativo, que determina cual será el estado final de los nodos de salida. De esta manera  $voutp$  se va alto si  $vinp$  es mayor que  $vinn$  y viceversa. Este tiempo se llama tiempo de comparación.

### 2.3.1. Presupuesto de Tiempos

Durante el proceso de conversión, se van generando 1 a 1 los bits del código de salida, el tiempo que tarda en decidirse cada bit se denomina tiempo de bit. El tiempo de conversión, será entonces  $N$  veces el tiempo de bit, siendo  $N$  el número de bits del código.

#### Ciclo de Conversión

Cuando la señal pulse esta en bajo, el comparador realiza la decisión, mientras que durante el tiempo alto la lógica genera el bit de acuerdo a la comparación y de acuerdo a esto el CDAC ajusta su salida.

En este caso, el tiempo bajo será variable, ya que por la naturaleza del comparador las entradas diferenciales mas pequeñas tomarán mas tiempo en resolverse. En cambio el tiempo alto será fijo debido al generador de pulsos, este tiempo está ajustado de manera de asegurar el settling del DAC antes del flanco de bajada. Llamaremos tiempo de settling del CDAC ( $T_{settlingCDAC}$ ) al tiempo desde que  $clk$  sube al 50 % de VDD hasta que las salidas del CDAC alcanzan el 90 % de su valor final.

Llamaremos tiempo de decisión ( $T_{decision}$ ) al tiempo desde que  $clk$  baja al 50 % de VDD hasta que el latch sale de metaestabilidad.

- Fase de reset (tiempo alto): Reset del comparador y settling del CDAC
- Fase de comparación (tiempo bajo)

El presupuesto de tiempos se diseñó considerando:

$$T_{bit} = T_{reset} + T_{comp} > T_{settlingCDAC} + T_{decision} \quad (2.10)$$

El generador de pulsos asegura esto a través de: Un tiempo de reset del doble del tiempo estimado de Settling y un tiempo de comparación que termina cuando alguna de las salidas del comparador va a alto, en este tiempo se incluyen entonces el retardo de los buffers y el retardo propio del generador de pulsos.

### 2.3.2. Arquitectura del Comparador

El comparador se basa en el diseño de polarización dinámica propuesto por Bindra [5] indicado en la figura 2.18 y la figura 2.19, modificando la arquitectura de Elzakker[4] para reducir consumo mediante un capacitor en la cola del preamplificador. Los tamaños se indican en la tabla 2.5

Se incluye un generador de pulsos que detecta cuando se realizó la comparación, y genera un tiempo de reset fijo para esperar el settling del DAC. El diagrama del mismo se indica en la figura 2.23. En el mismo se incluye un buffer Schmitt Trigger implementado como se indica en la figura 2.24.

El funcionamiento se ilustra en la figura 2.21 y se muestra su tiempo de reset en la tabla 2.6.

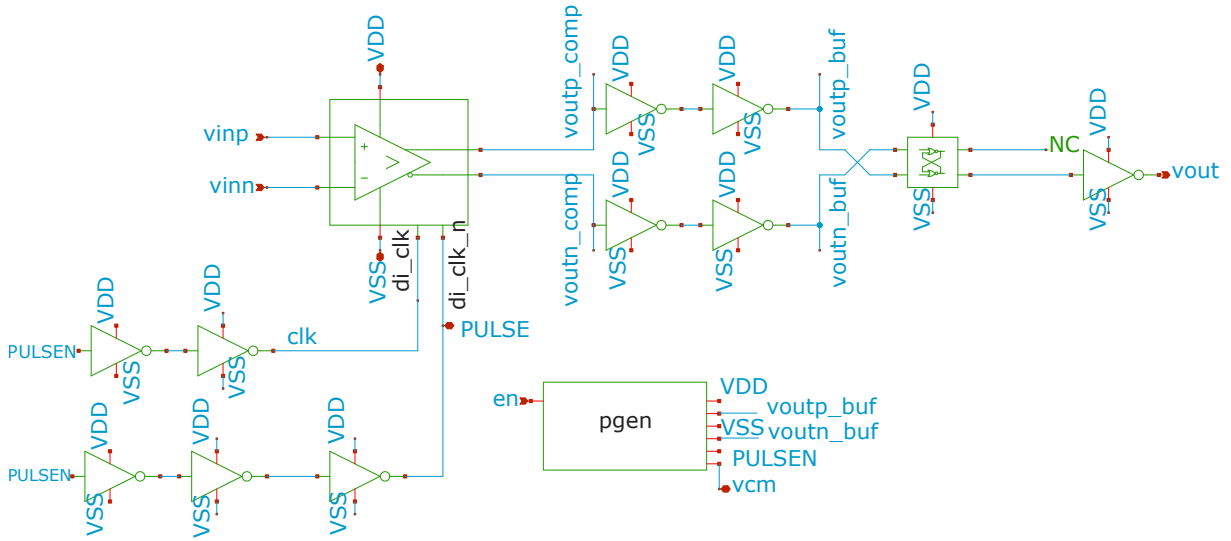


Figura 2.17: Diagrama de Bloques del comparador usado

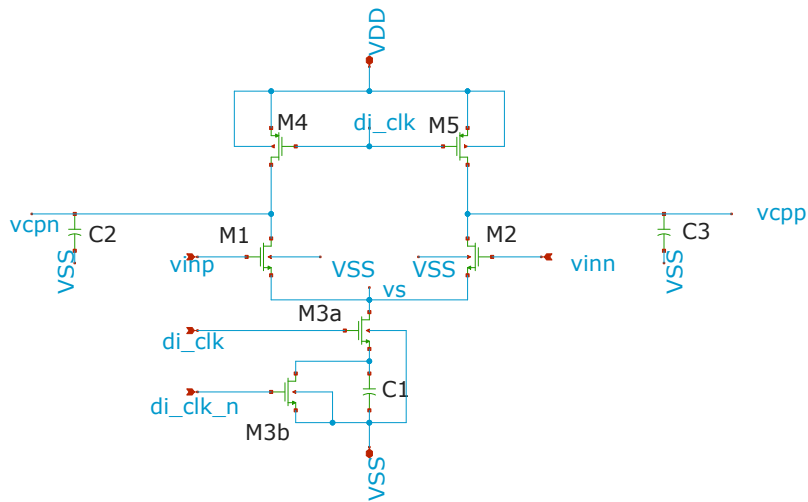


Figura 2.18: Preamplificador con Capacitor de Cola

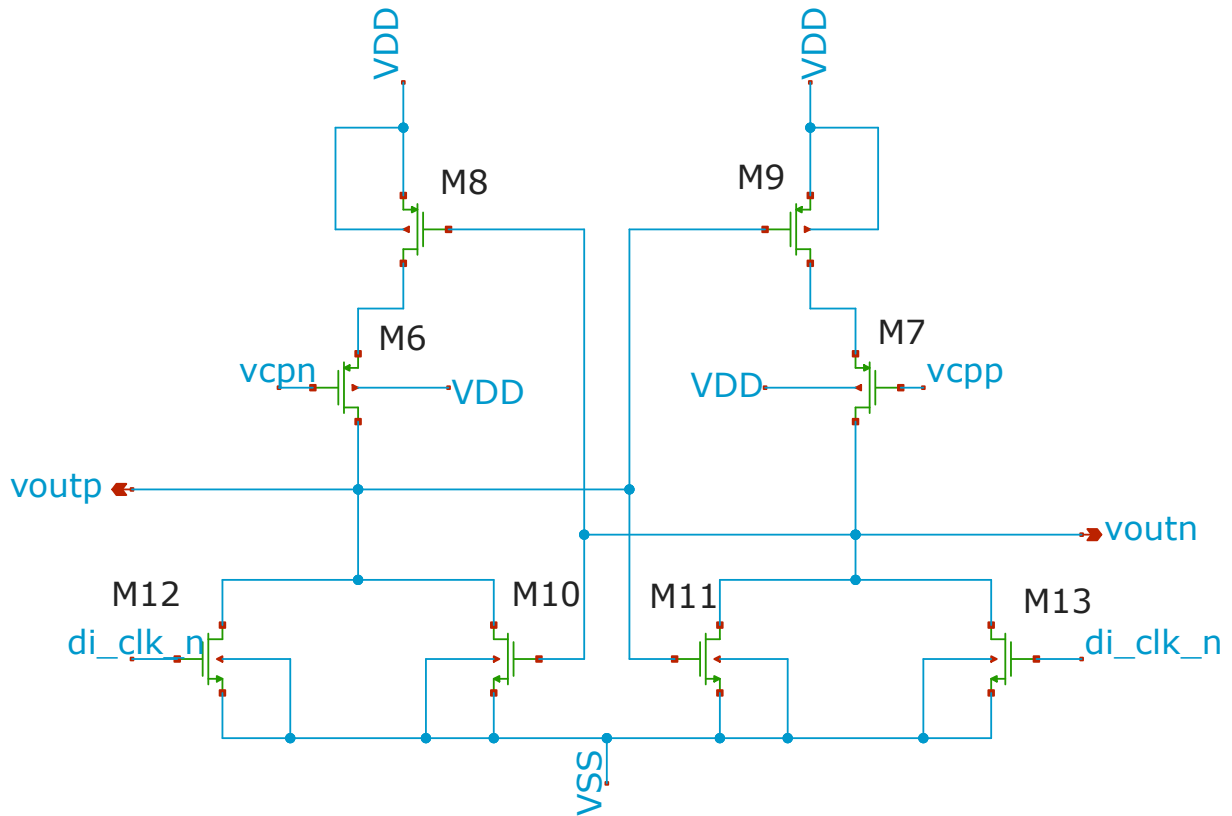


Figura 2.19: Latch del comparador Doble Cola

## Dimensionamiento

El diseño se basó en los valores reportados por Simon Dorrer [9], ajustados para la tecnología ihp-sg13g2. Se achicaron los capacitores de salida y agrandó el de cola para conseguir un menor tiempo de comparación, se agrandaron también los transistores de ambos pares (el del preamp y el latch) para aumentar la transconductancia, finalmente agrandamos un poco los transistores de reset (M4 y M5 para el preamp, M8 y M9 para el latch) para asegurar un correcto reset, evitando offset dinamico (cuando el resultado anterior sesga el próximo. El numero de gates en paralelo (ng) y el multiplicador (m) se ajustaron para lograr un diseño de layout mas cómodo. El  $W_{efectivo} = w * m$  el número de gates no modifica el  $w_{efectivo}$ , sino que  $w_{gate} = w_{efectivo}/ng$ . Para un modelo de primer orden solo usamos el  $w_{efectivo}$ , pero efectos de segundo orden, como capacidad de Miller dependen de la geometría implementada y debemos tener en cuenta los valores de ng y m.

### 2.3.3. Testbench del Comparador

#### Objetivo

A través del testbench de la figura 2.20 se busca caracterizar:

1. Tiempo de comparación para entrada de 1 LSB
2. Tiempo de reset requerido

Cuadro 2.5: Parámetros de diseño del comparador de doble cola tipo Bindra

| Dispositivo                         | Tipo | $W$ ( $\mu\text{m}$ ) | $L$ ( $\mu\text{m}$ ) | $ng$ | $m$ | Función/Nodo                         |
|-------------------------------------|------|-----------------------|-----------------------|------|-----|--------------------------------------|
| <b>Etapas de Preamplificación</b>   |      |                       |                       |      |     |                                      |
| M1                                  | NMOS | 5,0                   | 0.13                  | 1    | 1   | Par diferencial (vinp)               |
| M2                                  | NMOS | 5,0                   | 0.13                  | 1    | 1   | Par diferencial (vinn)               |
| M3a                                 | NMOS | 8,0                   | 0.13                  | 1    | 2   | Tail current source (parte a)        |
| M3b                                 | NMOS | 2,0                   | 0.13                  | 1    | 1   | Tail current source (parte b)        |
| M4                                  | PMOS | 3,0                   | 0.13                  | 1    | 1   | Reset(vcpp)                          |
| M5                                  | PMOS | 3,0                   | 0.13                  | 1    | 1   | Reset(vcpn)                          |
| <b>Etapas de Latch Regenerativo</b> |      |                       |                       |      |     |                                      |
| M6                                  | PMOS | 6,0                   | 0.13                  | 2    | 1   | Entrada diferencial latch (vcpn)     |
| M7                                  | PMOS | 6,0                   | 0.13                  | 2    | 1   | Entrada diferencial latch (vcpp)     |
| M8                                  | PMOS | 4,0                   | 0.13                  | 2    | 1   | Realimentación cruzada (voutp) [VDD] |
| M9                                  | PMOS | 4,0                   | 0.13                  | 2    | 1   | Realimentación cruzada (voutn) [VDD] |
| M10                                 | NMOS | 2,0                   | 0.13                  | 1    | 1   | Realimentación cruzada (voutp) [VSS] |
| M11                                 | NMOS | 2,0                   | 0.13                  | 1    | 1   | Realimentación cruzada (voutn) [VSS] |
| M12                                 | NMOS | 2,0                   | 0.13                  | 1    | 1   | Reset latch (voutp)                  |
| M13                                 | NMOS | 2,0                   | 0.13                  | 1    | 1   | Reset latch (voutn)                  |
| <b>Capacitores</b>                  |      |                       |                       |      |     |                                      |
| C1                                  | MIM  | 20,0                  | 20,0                  | –    | 1   | Capacitor de cola (dynamic bias)     |
| C2                                  | MIM  | 7,0                   | 7,0                   | –    | 1   | Vcpn                                 |
| C3                                  | MIM  | 7,0                   | 7,0                   | –    | 1   | Vcpp                                 |

Nota:  $ng$  = número de gates en paralelo,  $m$  = multiplicador

## Configuración del Test

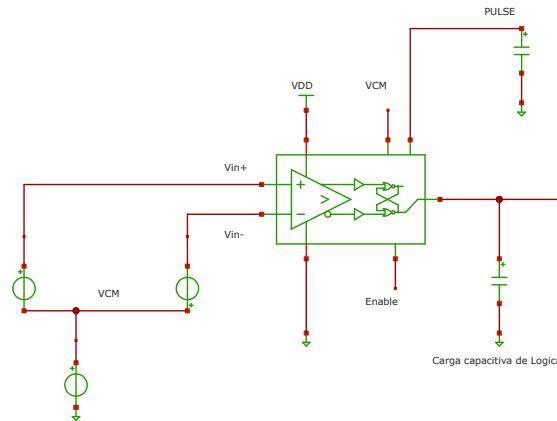


Figura 2.20: Testbench comparador completo

- Entrada diferencial:  $V_{IN} = \pm 0,5 \text{ LSB} = \pm 219 \text{ uV}$
- Mediciones: Tiempo hasta decisión estable, consumo promedio

## Resultados

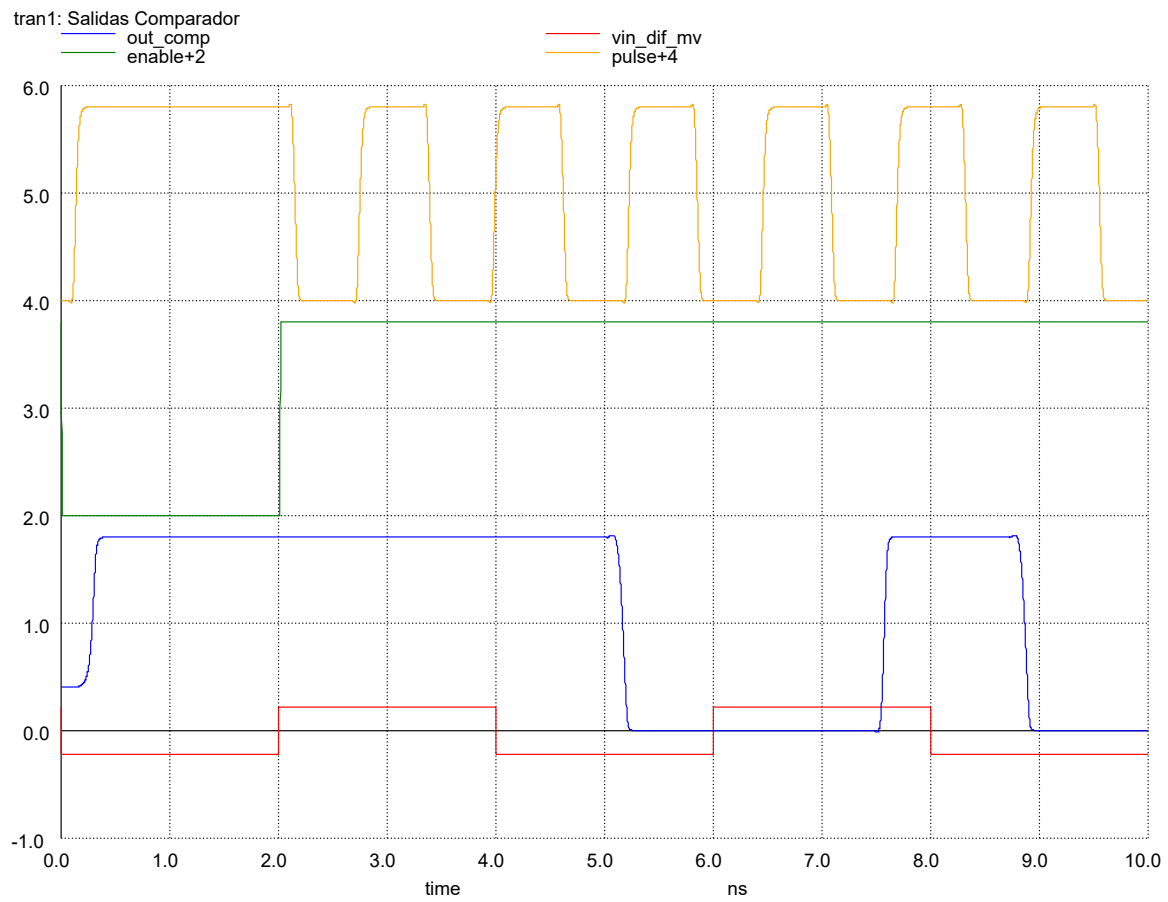


Figura 2.21: Señales de salida del comparador

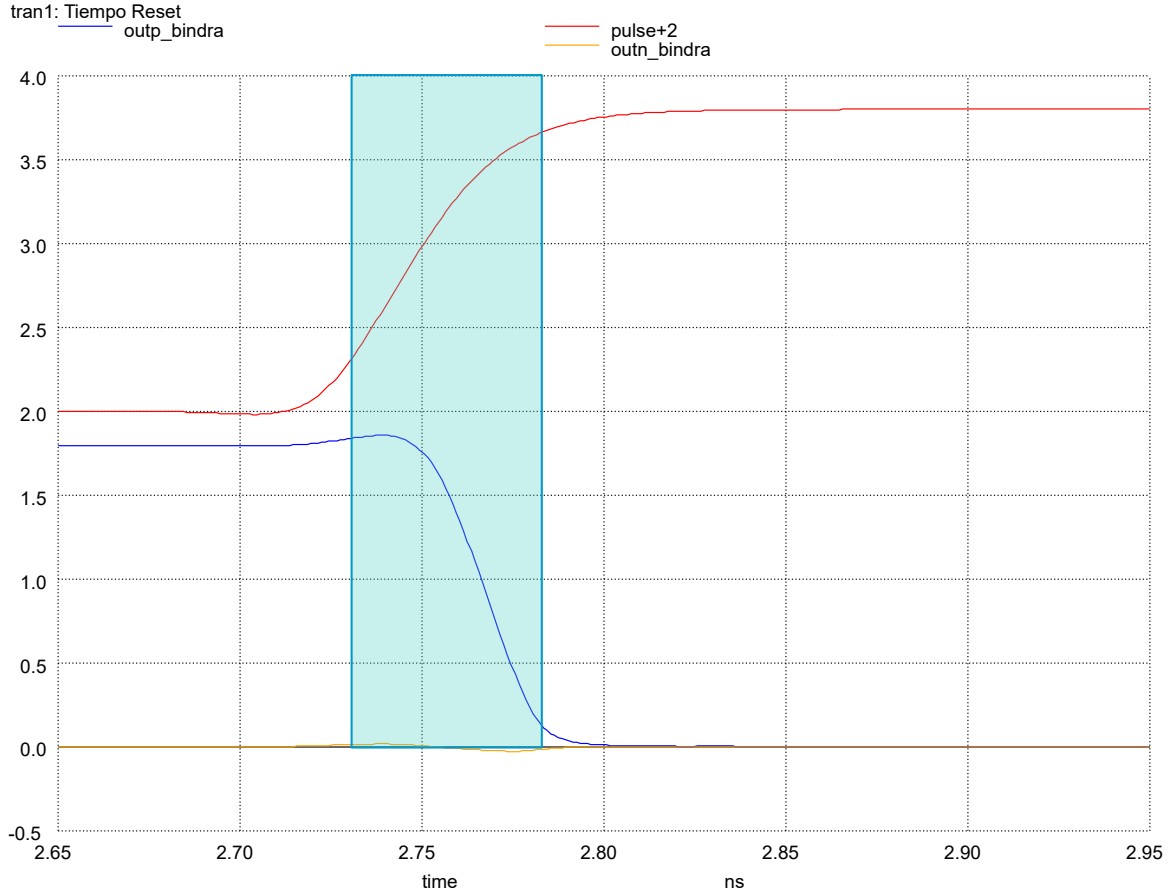


Figura 2.22: Tiempo de reset del comparador

### 2.3.4. Generador de Pulsos (Delay Line)

El generador de pulsos implementa el esquema de auto-clock del comparador y sincroniza la lógica SAR. El objetivo crítico es garantizar que el pulso de control se genere **después** de que el CDAC haya alcanzado su valor de settling.

#### Criterio de Diseño

El tiempo de reset del comparador debe ser aproximadamente:

$$T_{reset} \approx 2 \times T_{settling\_CDAC} \quad (2.11)$$

Esto asegura que:

- El CDAC tiene tiempo suficiente para establecerse
- El comparador puede resetear completamente
- Hay margen para variaciones de proceso

#### Implementación del generador de pulsos

El generador de pulsos se implementó mediante la carga del nodo vx a través del transistor M7, el cual tiene un largo mayor al mínimo (ajustado mediante iteraciones) y

limita la velocidad de carga del nodo vx, de esta forma aseguramos un tiempo de reset suficientemente largo para que el CDAC establezca su salida correctamente. Se puede observar las formas de onda obtenidas en la figura 2.25.

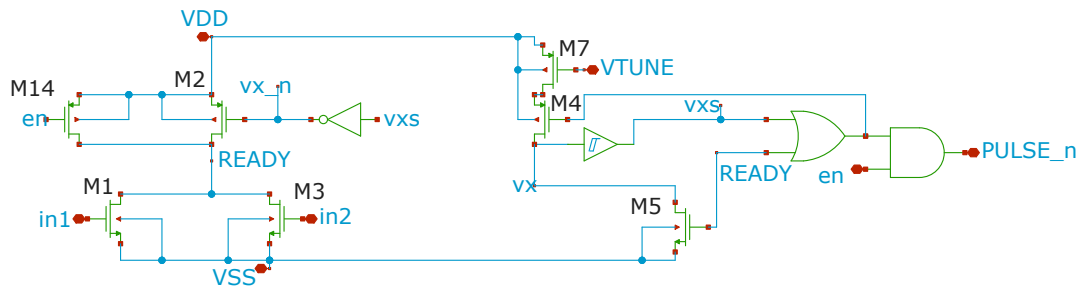


Figura 2.23: Diagrama del generador de pulsos

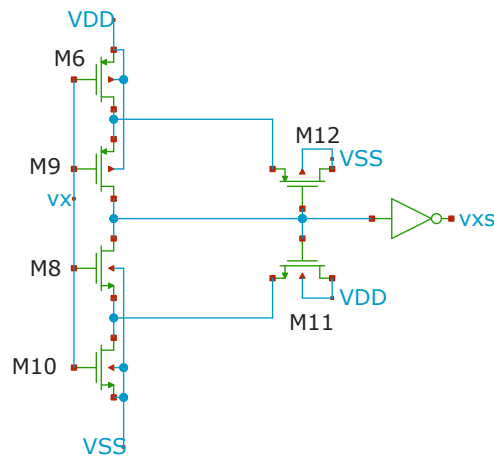


Figura 2.24: Implementación de la compuerta Schmitt-Trigger

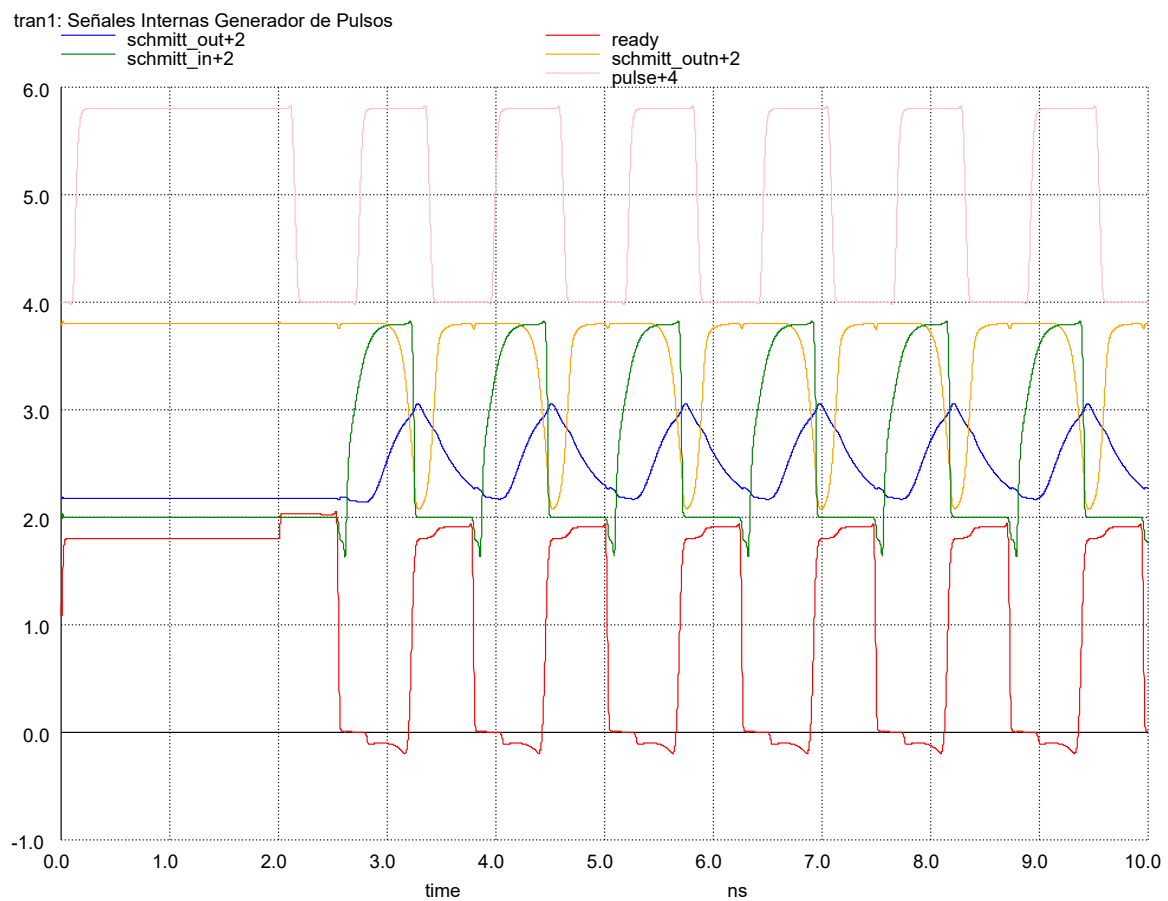


Figura 2.25: Formas de onda del generador de Pulsos

Cuadro 2.6: Resultados

| Parámetro                     | Valor  |
|-------------------------------|--------|
| Tiempo de comparación (1 LSB) | 548 ps |
| Tiempo de reset pgen          | 768 ps |
| Tiempo de reset comparador    | 56 ps  |

## 2.4. Buffers de Control

### Metodología de Cálculo de Buffers

El dimensionamiento de todas las cadenas de buffers se realizó siguiendo la metodología presentada en *Digital Integrated Circuits—A Design Perspective* de Rabaey [11]. Se procedió como se indica al principio para calcular  $C_{LOAD}$ , notar que si calculamos un numero no entero de cadenas (lo mas probable) procedemos a quedarnos con

- El que nos de la fase que queremos (Invertida o no)
- El que nos de el menor delay, tener en cuenta que el fan out debe agrandarse bastante mas de 4, para compensar el retardo de un inversor extra.
- Tener en cuenta que es mejor dejar el exceso de fanout para las ultimas etapas para minimizar el consumo agregado

### Testebench fanout optimo

Como se muestra en la la figura 2.26 al variar el fanout de una compuerta, varía tambien su retardo, se aplico la teoría clásica para buscar el fanout óptimo para una cadena de Buffers. El la tabla 2.7 indica los delays obtenidos y las la figura 2.27 la figura 2.28 muestran el fanout optimo estimado usando Geogebra.

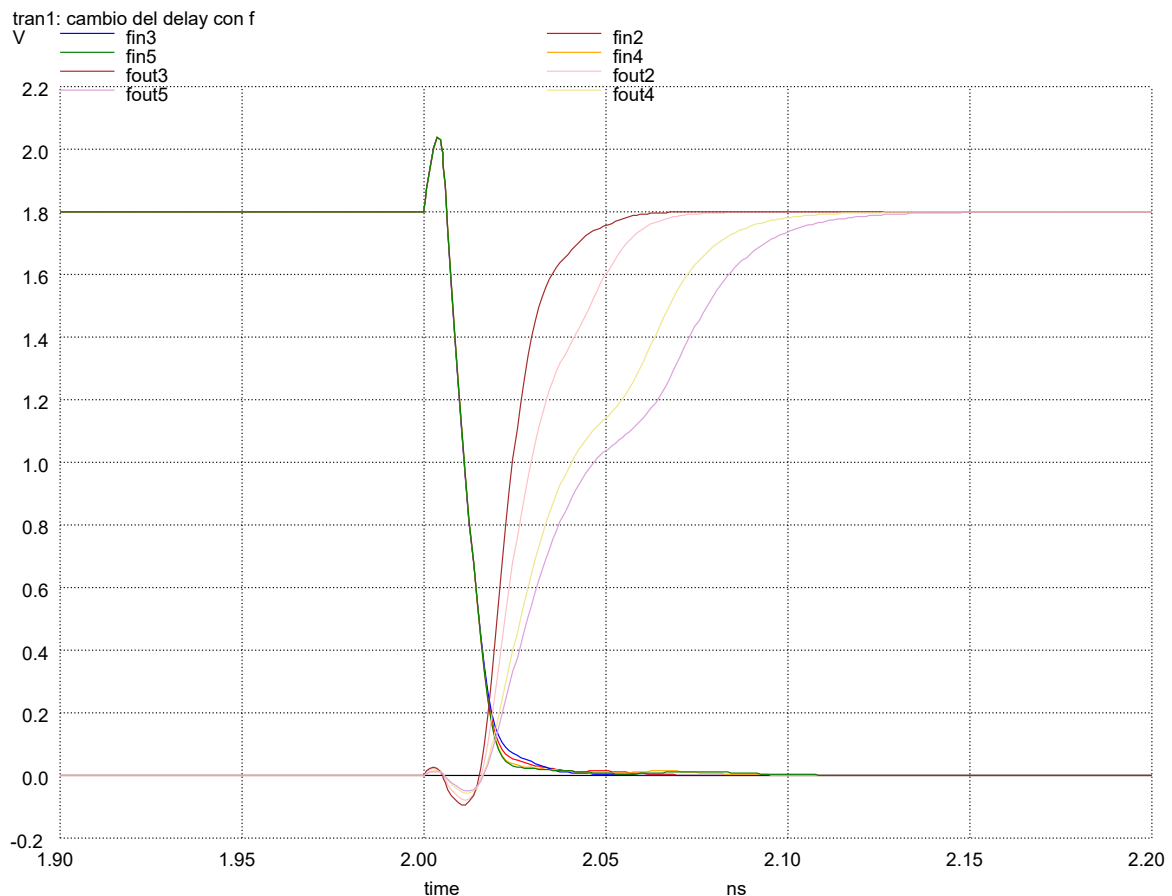


Figura 2.26: Retardos correspondiente a un fanout de 2, 3, 4 y 5

El modelo de delay de propagación de Rabaey para una cadena de buffers está dado por:

$$t_p = kR_W C_{in} \left( 1 + \frac{C_L}{C_{in}} \right) = t_{inv} (\gamma + f) \quad (2.12)$$

donde:

- $k$ : constante dependiente de la tecnología
- $R_W$ : resistencia efectiva del transistor
- $C_{in}$ : capacitancia de entrada del inversor
- $C_L$ : capacitancia de carga
- $f = C_L/C_{in}$ : fanout eléctrico
- $\gamma$ : parámetro de esfuerzo intrínseco
- $t_{inv}$ : delay intrínseco del inversor sin carga

Podemos aproximar el valor de  $\gamma$  como el cociente entre el tiempo de propagación de un inversor descargado dividido en la derivada del tiempo de propagación respecto del layout (en este caso el cambio al aumentar el fan out en 1):

$$\gamma = \frac{t_{p0}}{t_{p'inv}} \quad (2.13)$$

. Podemos observar que este valor no es constante y depende del fanout, de la literatura sabemos que el fanout óptimo está alrededor de 4, por lo que procedemos a estimar el parámetro gamma con los valores correspondientes al fanout 4.

Usando Geogebra podemos derivar el retardo de una cadena de buffers y encontrar el mínimo del retardo para un cierto número de etapas, el fanout será entonces la raíz n-esima del fan out total que ingresamos, si repetimos para varios valores, encontramos un valor relativamente constante para el fanout. Procedemos luego a realizar pruebas con los valores enteros más próximos, de manera que podemos encontrar un fanout óptimo entero; ya que implementaremos las cadenas de buffer con multiplicadores. Delay del inversor en vacío es  $9ps$ . El valor encontrado es: 1.5 para el parámetro gamma, lo que nos lleva

Cuadro 2.7: Análisis de delay vs fanout para optimización de cadenas de buffers

| Fanout | Delay (ps) | $\Delta$ Delay (ps) |                           |
|--------|------------|---------------------|---------------------------|
| 2      | 18ps       | –                   | Referencia                |
| 3      | 23ps       | 5ps                 |                           |
| 4      | 29ps       | 6ps                 | <i>Optimo segun libro</i> |
| 5      | 35ps       | 6ps                 |                           |

a un fanout óptimo de 4 efectivamente en geogebra. En la curva azul, el delay estimado para un número de etapas  $x$ , en la gráfica gris el fanout óptimo para el número de etapas  $x$ . La curva verde resalta el fanout óptimo del número de etapas mínimo.

Podemos entonces, encontrar el número de etapas como

$$N = \ln(C_{load}/C_{in})/\ln(4) = \log_4(C_{load}/C_{in})$$

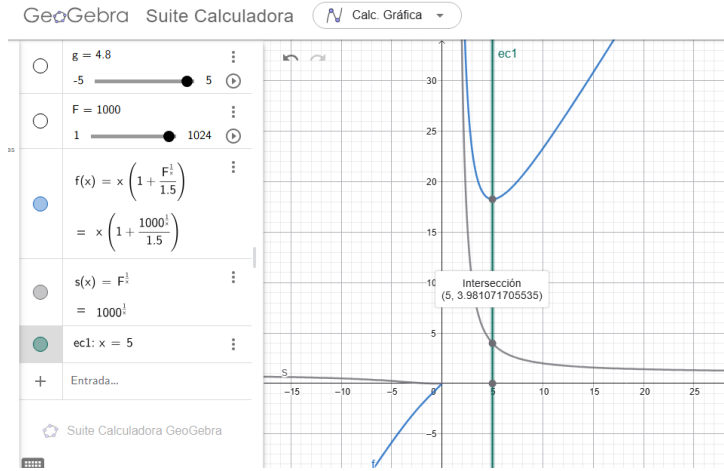


Figura 2.27: Valor del fanout f optimo para un fanout total F=1000

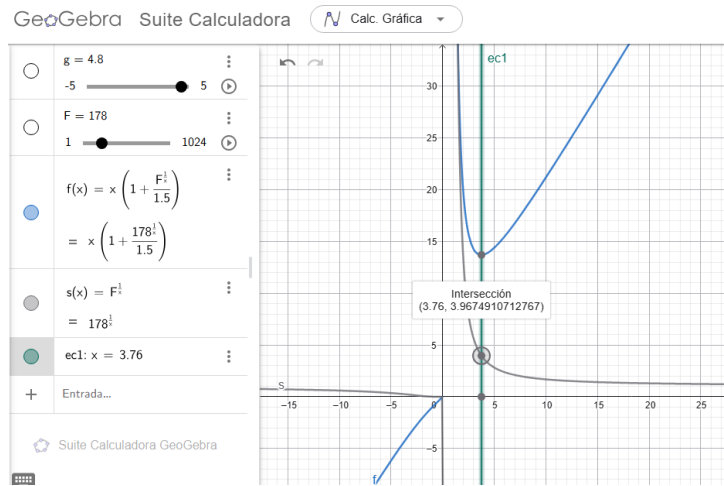


Figura 2.28: Valor del fanout f optimo para otro fanout total F=178

El sistema requiere varias cadenas de buffers para manejar las cargas capacitivas de los diferentes bloques. A continuación se indica como se encontraron las cargas capacitivas a manejar. Se consideró como inversor mínimo, un con  $W_n = 0,15\mu m$  y  $W_p = 0,3\mu m$  además  $L_n = L_p = L_{min} = 0,13\mu m$  por lo que para calcular la  $C_{LOAD}$  se procedió de la siguiente manera, aproximando las capacitancias de compuerta de ambos MOS al mismo valor, (suponemos esto debido a las similitudes de los valores reportados de CJUNCTION y CMILLER proporcionados por la documentación del proceso) [8]:

- Multiplica  $W$  por  $m$  de cada transistor conectado al nodo, y luego se suma todo.
- Se divide el valor en  $0,45\mu m$  para obtener la capacidad del nodo relativa a la capacidad de entrada de un inversor unitario

### Buffer de Salidas del Comparador

La salida del comparador maneja el latch que retiene la última comparación. Se implementó una cadena de buffers para:

- Aislar el nodo de salida sensible del comparador, esto evita introducir offset y degradar demasiado el tiempo de conversión por el estado del latch. En el nodo de salida se conecta el inversor unitario
- Proveer drive suficiente para el latch.

En la figura 2.29 se muestran los retardos obtenidos. En este caso de 11ps desde que el comparador toma la decision (cuando una salida empieza a bajar) hasta el 50 % de VDD en la salida del buffer,.

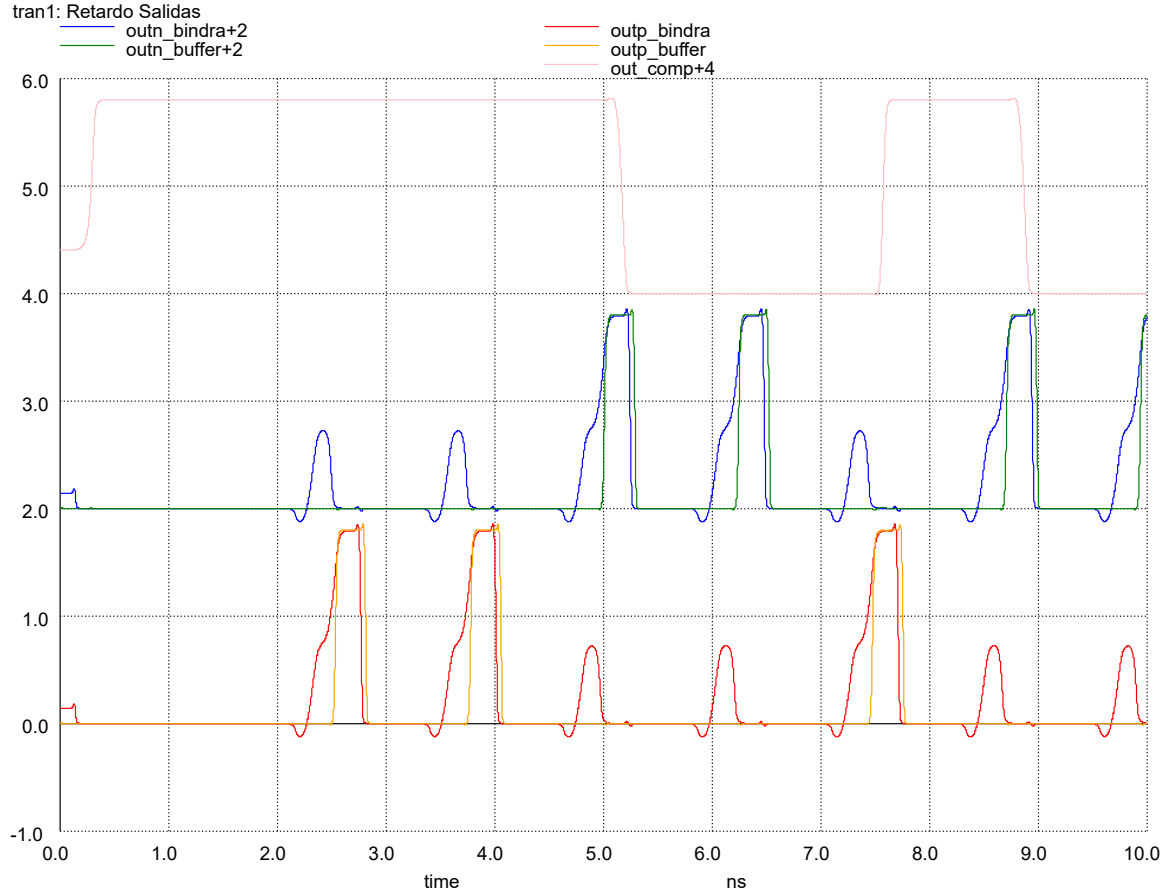


Figura 2.29: Retardo del buffer de la salida del comparador hacia el latch

$$F_{LOAD} = 24$$

### Buffer de Salida Pulse del Comparador

La salida que implementa el reloj del comparador mueve varios nodos

- Señales di\_clk y di\_clk.n del preamp y el latch
- Señal de reloj de la lógica
- di\_clk.n y reloj logica  $C_{LOAD} = 218$
- di\_clk  $C_{LOAD} = 48$

## Buffer de Señales de Lógica a CDAC

Las señales de control digitales ( $D[n]$  y  $B[n]$ ) provenientes de la lógica SAR deben manejar todos los switches del CDAC. Se implementó una cadena de buffers para garantizar tiempos de rise/fall adecuados. Por cada transistor pmos unitario son  $4.44 C_{inv}$  y el nmos  $2.22 C_{inv}$  aproximadamente. tendremos entonces 3 buffers diferentes a implementar.

$$F_{TG_{nmos}} = n * 2,22$$

$$F_{TG_{pmos}} = n * 4,44$$

$$F_{nmos} = n * 2,22$$

$$F_{pmos} = n * 4,44$$

## Buffer de Señales de Sampleo

Las señales non-overlap generadas deben manejar la carga capacitiva de todos los switches del array de muestreo. Para garantizar tiempos de rise/fall adecuados y evitar degradación de las señales, se implementó una cadena de buffers. De forma similar al caso anterior, pero esta vez con un numero fijo de transistores por nodo. Tenemos 2 dac (p y n) con 2 subdac, cada uno con 63 transistores, además tengamos en cuenta que la señal B maneja compuertas con dummies asi que agregamos sus capacitades.

- En el nodo Sample A se conectan 252 nmos
- En el nodo Sample B se conectan 378 pmos (por los dummies)
- En el nodo Sample A negado se conectan 252 pmos
- En el nodo Sample B negado se conectan 378 nmos (por los dummies)

$$F_{LOADsampleA} = 252 * 2,2 = 554,4$$

$$F_{LOADsampleB} = 378 * 4,4 = 1663$$

$$F_{LOADsampleAn} = 252 * 4,4 = 1108$$

$$F_{LOADsampleBn} = 378 * 2,2 = 831$$

Cuadro 2.8: Diseño de cadenas de buffers - Cálculo de etapas y multiplicadores

| $F_{LOAD}$ | Número de etapas<br>(cálculo) | Número de<br>Etapas | Multiplicadores |
|------------|-------------------------------|---------------------|-----------------|
| 24         | 2.29248125                    | 2                   | 4-8             |
| 48         | 2.79248125                    | 2                   | 4-8             |
| 218        | 3.88409162                    | 3                   | 4-8-32          |
| 554        | 4.56687083                    | 4                   | 4-8-32-64       |
| 1663       | 5.34978627                    | 5                   | 4-8-32-64-256   |
| 1108       | 5.05687083                    | 5                   | 4-8-32-64-256   |
| 831        | 4.84935233                    | 4                   | 4-8-32-64       |
| 4.44       | 1.07527983                    | 1                   | —               |
| 8.88       | 1.57527983                    | 1                   | —               |
| 17.76      | 2.07527983                    | 2                   | 4-8             |
| 35.52      | 2.57527983                    | 2                   | 4-8             |
| 71.04      | 3.07527983                    | 3                   | 4-8-32          |
| 8.88       | 1.57527983                    | 1                   | —               |
| 17.76      | 2.07527983                    | 2                   | 4-8             |
| 35.52      | 2.57527983                    | 2                   | 4-8             |
| 71.04      | 3.07527983                    | 3                   | 4-8-32          |
| 142.08     | 3.57527983                    | 3                   | 4-8-32          |

## 2.5. Lógica SAR

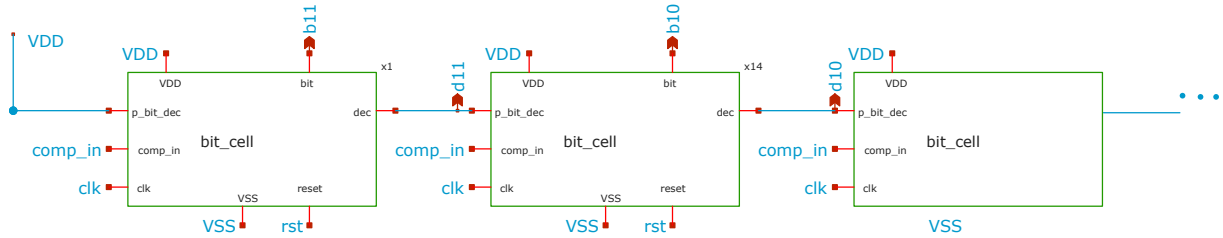


Figura 2.30: Fragmento de la cadena de la lógica implementada

La lógica SAR controla la secuencia de conversión y almacena los bits resueltos. Se implementó de manera síncrona utilizando celdas estándar del PDK ihp-sg13g2 formando una cadena "dominó" que activa la señal de decidido y retiene el bit a medida que recibe un 1 (desde  $D[n-1]$ ) de la celda anterior, como se indica en la figura 2.30 mostrando solo el principio de esta cadena que termina con la celda que maneja  $D[0]$  y  $B[0]$ . Cada bloque unitario de la lógica consiste en 2 partes:

- El latch que guarda decidido, que propaga el valor lógico alto en cada pulso generado por la decisión del comparador. Genera la señal  $D[n]$
- El latch que retiene el valor de la decisión del comparador de ese momento. Genera la señal  $Bit[n]$

## 2.5.1. Bloque Unitario de Lógica

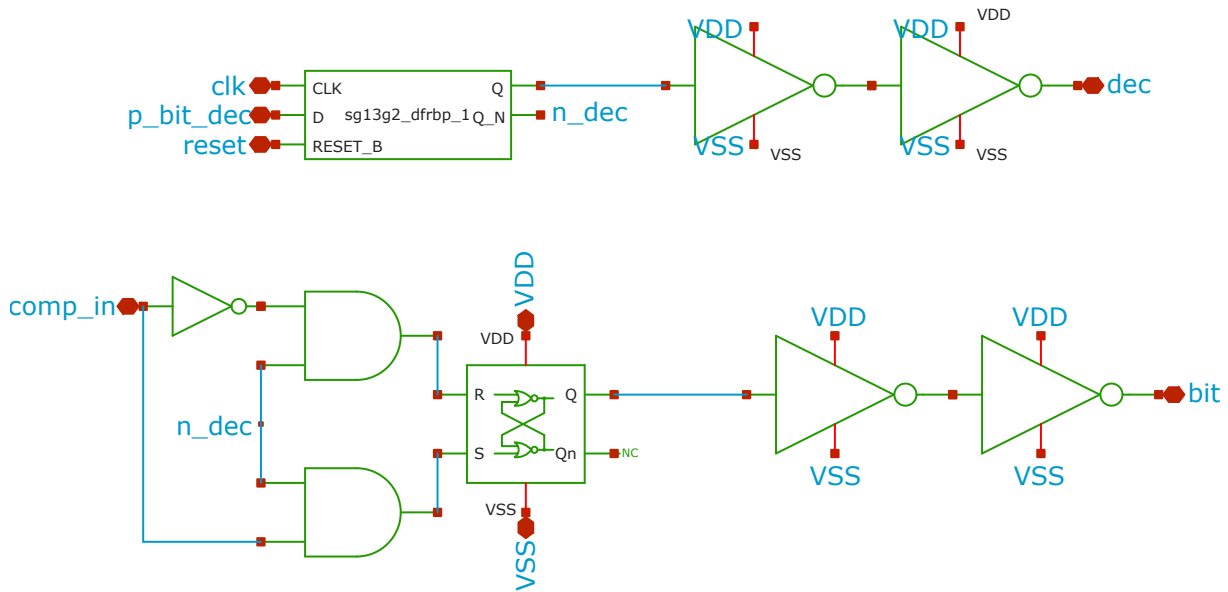


Figura 2.31: Bloque unitario de la lógica, genera el bit  $B[n]$  y la señal de decidido  $D[n]$

b En la figura 2.31 se muestra el diagrama de cada eslabón que forma la cadena "dominó" de la logica. La celda lógica sg13g2\_dfrbp\_1 implementa un flip-flop tipo D, activado por flanco con reset asincrónico activo bajo. La caracterizacion (tabla de verdad, delays) puede verse en: [8]

Cada bit de la conversión requiere un bloque de lógica que incluye:

- Registro para retener el bit de decidido  $D[n]$
- Biestable para retener el bit de control  $B[n]$
- Lógica combinacional para generar señales de control del CDAC (En la implementación Layout)
- Interfaz con el bit anterior  $D[n - 1]$  para propagar la habilitación

### Entradas y Salidas del Bloque Unitario

#### Entradas:

- Salida del comparador (decisión del bit actual)
- Pulso del generador de pulsos (reloj de la lógica)
- $D[n - 1]$  (habilitación desde el bit anterior)

### Entradas y Salidas del Bloque Unitario

#### Entradas:

- Salida del comparador (decisión del bit actual)

- Señal de Reset

**Salidas:**

- $D[n]$ : Señales de bits resueltos
- $B[n]$ : Bit de control para el CDAC/Código de Salida

### 2.5.2. Implementación con Celdas Estándar

La implementación se realizó utilizando latches y compuertas lógicas de la biblioteca de celdas estándar del PDK ihp-sg13g2. Esto simplifica el diseño y garantiza:

- Comportamiento lógico verificado
- Timing caracterizado
- Layout DRC-clean

### 2.5.3. Análisis de Timing

#### Delay de Propagación

El parámetro crítico es el delay desde el flanco de subida del generador de pulsos hasta el cambio en las señales  $D[n]$  y  $B[n]$  que controlan el CDAC.

Este delay debe ser suficientemente pequeño para que:

$$T_{bit} = T_{reset} + T_{logic} + T_{comp} < T_{periodo} \quad (2.14)$$

#### Medición del Delay

Se realizó simulación transitoria midiendo:

- Delay de flanco de subida del generador de pulsos al cambio en  $D[n]$
- Delay de flanco de subida del generador de pulsos al cambio en  $B[n]$
- Variación del delay a lo largo de la cadena (bit 1 vs bit 12)

Se midió el delay desde el flanco de subida de clk hasta el flanco de subida de las señales de la celda unitaria de lógica. Esto es porque ambas señales de salida siempre inician en 0. Se midió el delay en la simulación TOP, es decir con la lógica cargada.

Cuadro 2.9: Parámetros de Timing de la Lógica SAR

| Parámetro                      | Valor  |
|--------------------------------|--------|
| Delay pulso $\rightarrow D[n]$ | 258 ps |
| Delay pulso $\rightarrow B[n]$ | 115 ps |

## 2.6. Simulación TOP

Se realizaron simulaciones tran a nivel TOP, usando entradas DC para probar la funcionalidad del circuito. Se muestra el testbench utilizado en la figura 2.1 mientras que las salidas obtenidas pueden verse en la figura 2.33 y el proceso de aproximaciones sucesivas en la salida del dac se ilustra en la figura 2.32. Finalmente podemos ver como la entrada del comparador decrece monotonamente en la figura 2.34

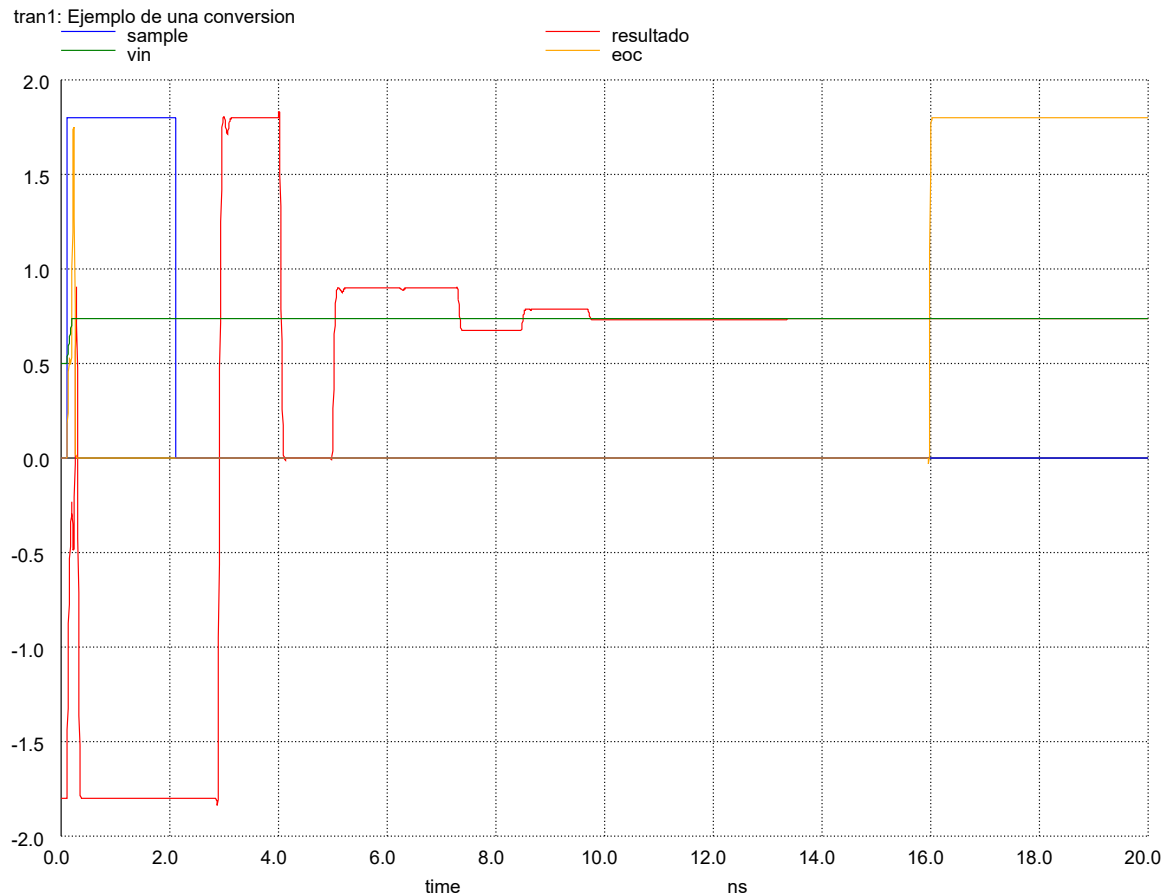


Figura 2.32: Ejemplo de una conversión completa, se observa como el resultado se va a aproximando al valor de la entrada

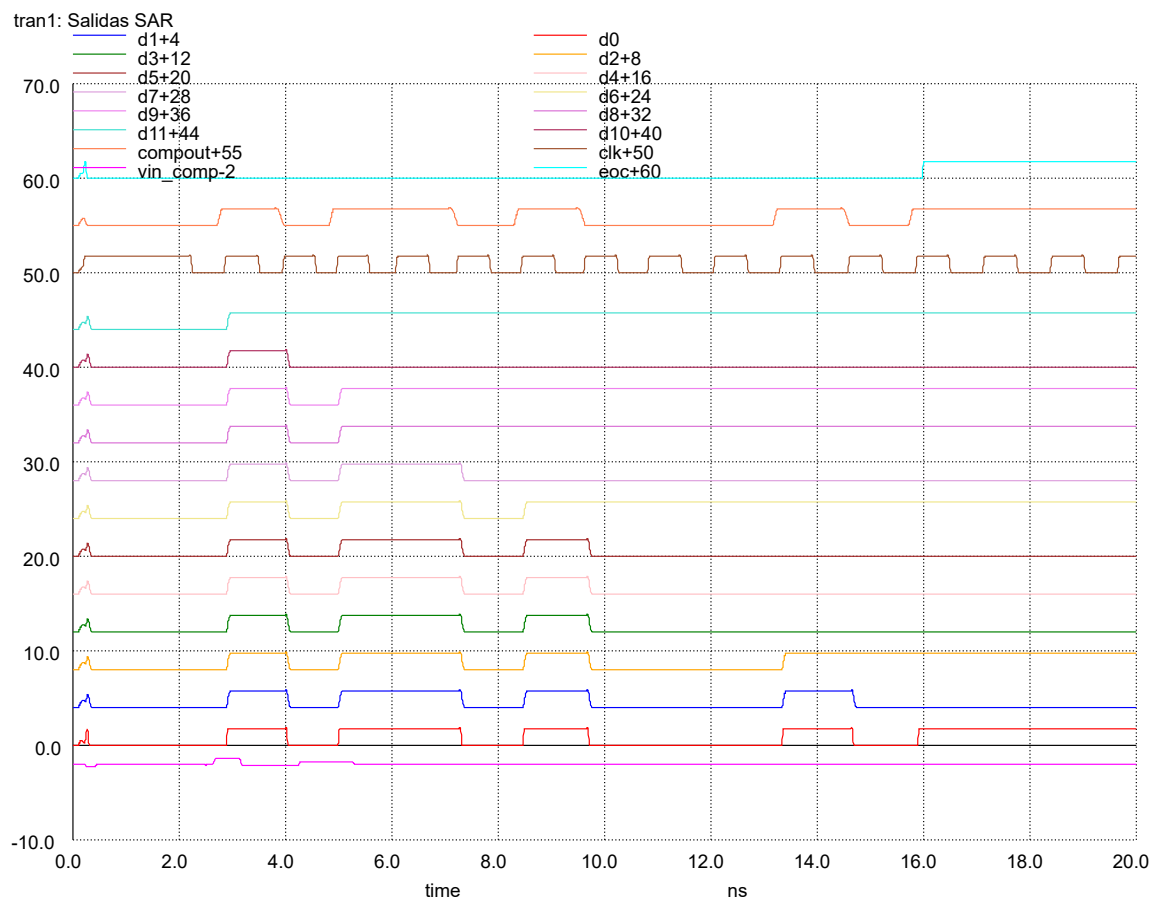


Figura 2.33: Salidas correspondientes al ejemplo

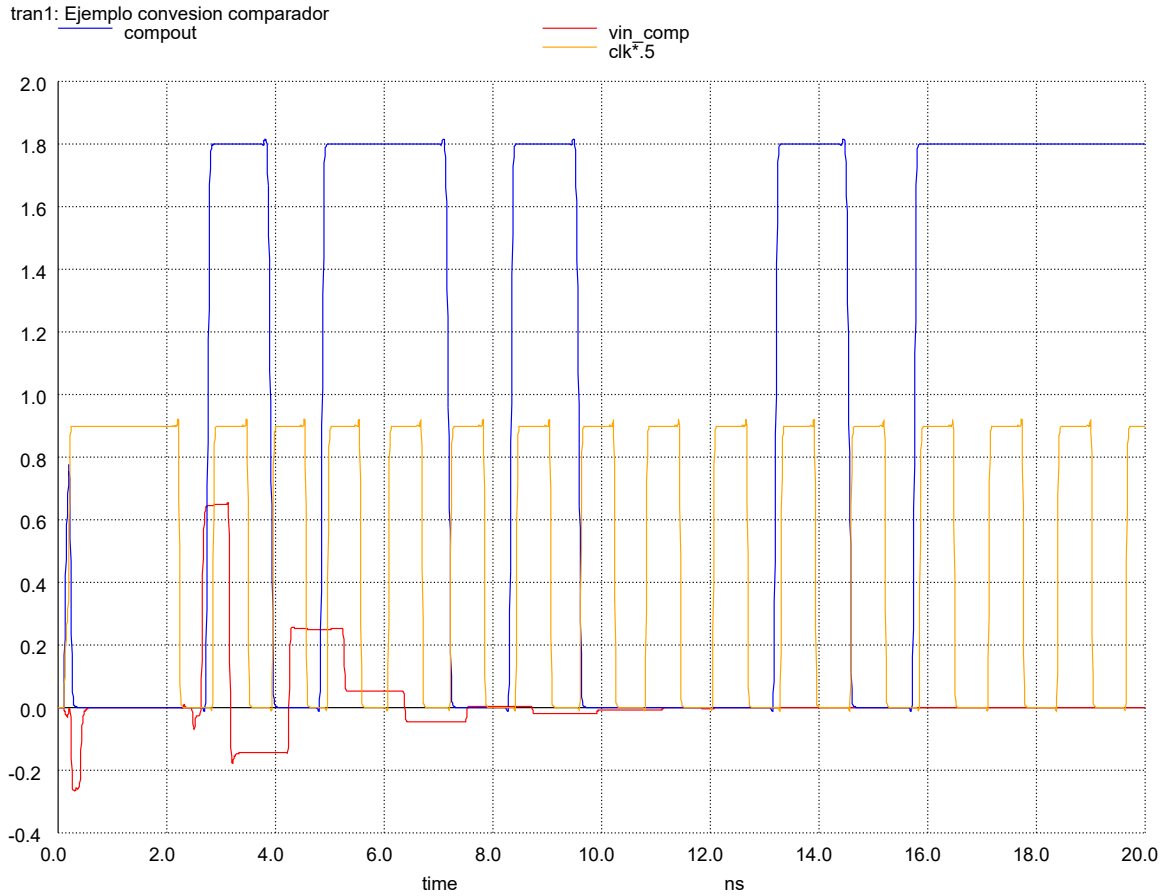


Figura 2.34: Entradas del comparador en el ejemplo (se escaló el clk para mejor visualización)

### 2.6.1. Caracterización

Se realizaron simulaciones con entrada DC para aproximar la longitud de los intervalos correspondientes a los códigos de  $V_{in} = 0$ ,  $V_{in} = V_{DD}$  y  $V_{in} = \frac{V_{DD}}{2}$ , para lo cual se simularon 4 valores en los supuestos intervalos entre el código en cuestión y el anterior/siguiente. Se detalla los valores obtenidos en la tabla 2.10:

Cuadro 2.10: Resultados de simulación: Verificación de códigos y transiciones (Valores en mV).

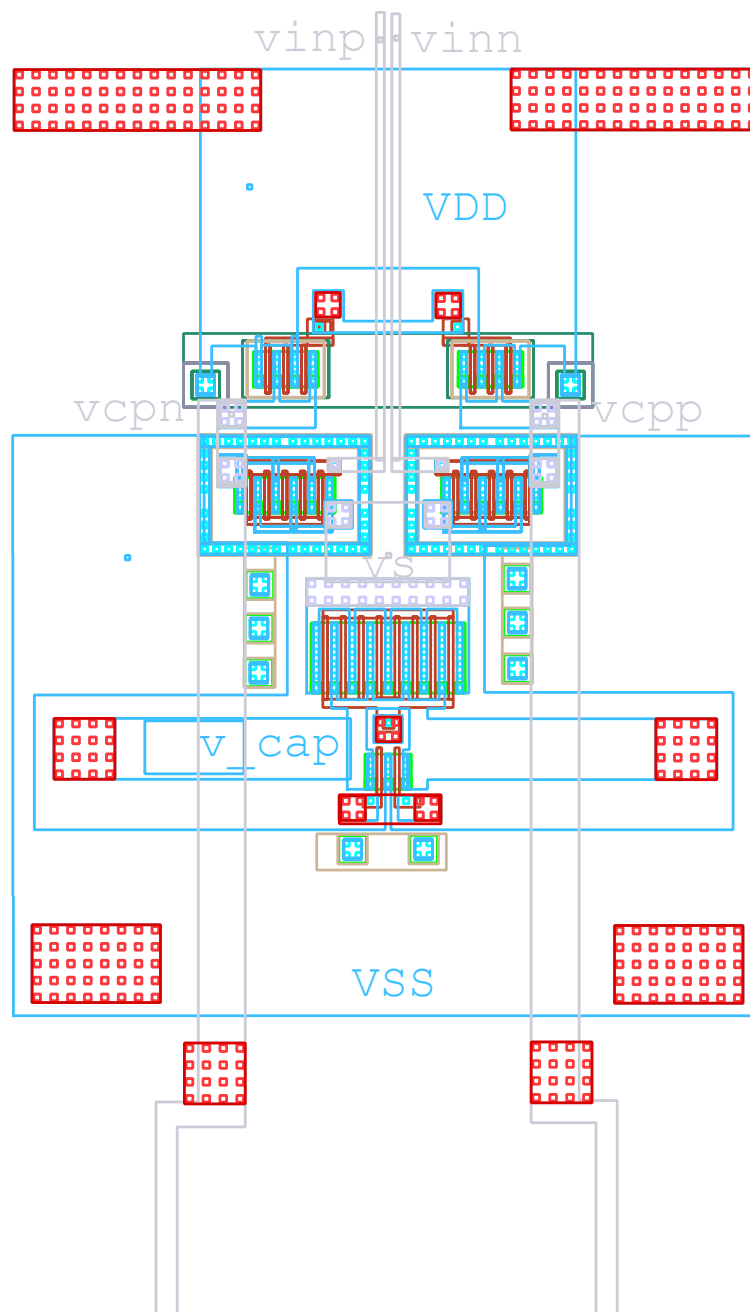
| Región      | Posición (LSB) | $V_{in}$ (mV) | Cód. Esperado    | Cód. Obtenido |
|-------------|----------------|---------------|------------------|---------------|
| Código 0    |                |               |                  |               |
| 4* (Inicio) | 0,125          | 0,109 863     | 0                | <b>0</b>      |
|             | 0,250          | 0,219 726     | 0                | <b>0</b>      |
|             | 0,375          | 0,329 589     | 0                | <b>0</b>      |
|             | 0,500          | 0,439 453     | 0/1 (Transición) | <b>0</b>      |
| Código 1024 |                |               |                  |               |
| 8* (Mitad)  | 1023,500       | 899,560 291   | 1023/1024        | <b>1023</b>   |
|             | 1023,625       | 899,670 154   | 1024             | <b>1024</b>   |
|             | 1023,750       | 899,780 017   | 1024             | <b>1024</b>   |
|             | 1023,875       | 899,889 880   | 1024             | <b>1024</b>   |
|             | 1024,125       | 900,109 607   | 1024             | <b>1024</b>   |
|             | 1024,250       | 900,219 470   | 1024             | <b>1024</b>   |
|             | 1024,375       | 900,329 333   | 1024             | <b>1024</b>   |
|             | 1024,500       | 900,439 197   | 1024/1025        | <b>1024</b>   |
| Código 2048 |                |               |                  |               |
| 4* (Final)  | 2047,500       | 1799,560 035  | 2047/2048        | <b>2047</b>   |
|             | 2047,625       | 1799,669 898  | 2048             | <b>2048</b>   |
|             | 2047,750       | 1799,779 762  | 2048             | <b>2048</b>   |
|             | 2047,875       | 1799,889 625  | 2048             | <b>2048</b>   |



# Capítulo 3

## Diseño de Layout

### 3.1. Preamplificador



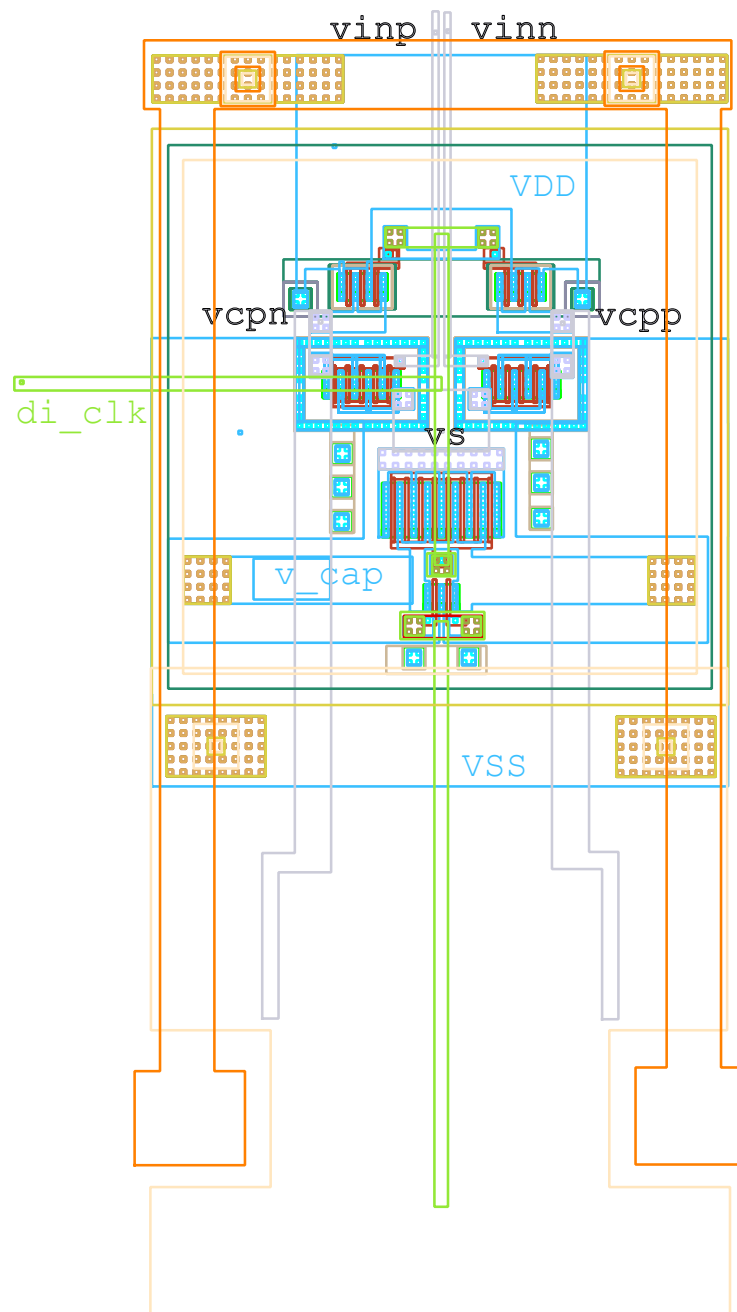


Figura 3.2: Layout del preamplificador, mostrando todas las capas

# Capítulo 4

## Conclusiones y Trabajo Futuro

### 4.1. Estado Actual del Trabajo

Este documento presenta la implementación de un ADC SAR de 12 bits utilizando herramientas de código abierto y la tecnología ihp-sg13g2. Al momento de esta versión del informe, el diseño esquemático y las simulaciones funcionales están completos, y se ha avanzado significativamente en el layout físico. Se han completado y verificado (DRC/LVS) los siguientes bloques: comparador, celda unitaria del CDAC, latch del comparador, y generador de pulsos (implementado con transistores NMOS en triple-well). Sin embargo, el layout completo del sistema integrado y las simulaciones post-layout están pendientes para la versión final de este trabajo.

### 4.2. Conclusiones

#### 4.2.1. Logros Principales

El presente trabajo ha logrado diseñar e implementar un ADC SAR de 12 bits funcional en tecnología ihp-sg13g2, cumpliendo con los objetivos establecidos:

- **Diseño Funcional Completo:** Se implementó exitosamente un ADC SAR de 12 bits basado en la arquitectura IMCS (Inverted Merged Capacitor Switching), incluyendo todos los bloques necesarios: CDAC split-binary (6+6 bits), circuito de Sample & Hold, comparador de doble cola tipo Bindra, generador de pulsos auto-sincronizado, y lógica SAR implementada con celdas estándar.
- **Rendimiento Alcanzado:** Las simulaciones funcionales demostraron que el diseño logra resolver conversiones dentro del error de cuantización esperado, con un tiempo de conversión promedio de aproximadamente 18 ns y un consumo energético de aproximadamente 220 pJ por conversión.
- **Verificación de Layout:** Los bloques de layout completados (comparador, celda unitaria del CDAC, latch del comparador, y generador de pulsos) pasaron exitosamente las verificaciones DRC (Design Rule Check) y LVS (Layout vs Schematic), demostrando la manufacturabilidad del diseño en el PDK ihp-sg13g2.
- **Documentación Completa:** Se logró el objetivo principal de proveer una guía didáctica exhaustiva del proceso de diseño de circuitos integrados analógicos utilizando exclusivamente herramientas de código abierto (xschem, ngspice, klayout).

Todo el código, esquemáticos, scripts y resultados están disponibles públicamente en el repositorio [1].

- **Especificaciones como Baseline:** Aunque este diseño no fue optimizado exhaustivamente, las métricas obtenidas pueden servir como especificaciones mínimas para futuros diseños más sistemáticos. Las siguientes métricas fueron caracterizadas:
  - Tiempo de conversión:  $\sim 18$  ns/conversión
  - Energía por conversión:  $\sim 220$  pJ/conversión
  - Figura de mérito (FoM):  $\sim 14$  pJ/conversión-paso (estimado)
  - Resolución funcional: 12 bits dentro del error de cuantización

### 4.2.2. Lecciones Aprendidas y Recomendaciones Metodológicas

Durante el desarrollo de este trabajo se identificaron varios aspectos metodológicos que pueden ser de utilidad para futuros diseñadores:

#### Simulaciones y Herramientas

Los tiempos de simulación prolongados complicaron significativamente el análisis de performance. En ocasiones, el simulador presentaba problemas de convergencia que requerían ajustes en las condiciones iniciales o en el paso de integración. **Recomendación:** Un mejor entendimiento y práctica con ngspice acelera enormemente el trabajo. En particular, aprender a utilizar los comandos `.measure` para extracción automática de mediciones y el modo interactivo de ngspice fueron puntos de inflexión que mejoraron sustancialmente la productividad.

El uso de foros especializados como FOSSI-chat resultó invaluable, permitiendo contacto directo con desarrolladores de las herramientas para resolver problemas específicos. **Recomendación:** Participar activamente en las comunidades de código abierto no solo acelera la resolución de problemas, sino que también contribuye al ecosistema general.

#### Documentación Continua

Un error metodológico importante fue dejar la documentación para las etapas finales del proyecto. Esto no solo dificultó recordar detalles específicos de las decisiones de diseño, sino que también privó al proceso de los beneficios de la escritura reflexiva. **Recomendación:** Escribir los resultados e ideas de manera continua es extremadamente útil para entender mejor lo que se está haciendo, identificar inconsistencias, y mantener un registro preciso de las decisiones de diseño y sus justificaciones.

## 4.3. Trabajo Futuro

### 4.3.1. Mejoras en Metodología y Automatización

#### Automatización de Simulaciones

Un punto de partida valioso para mejorar la eficiencia del flujo de diseño sería la automatización completa de las simulaciones, incluyendo:

- Scripts para ejecutar baterías de simulaciones paramétricas
- Extracción automática de métricas utilizando comandos `.measure`
- Ordenamiento automático de resultados, gráficos e imágenes
- Generación automática de reportes con las métricas extraídas

Esta automatización permitiría explorar el espacio de diseño de manera mucho más eficiente y facilitaría el análisis de sensibilidad paramétrica.

## Diseño Basado en Corners

Aunque no se puede modelar mismatch en simulaciones esquemáticas, sería extremadamente útil poder realizar simulaciones automáticas para diseñar sistemas que cumplan especificaciones en todos los corners de proceso, voltaje y temperatura (PVT). Esto permitiría garantizar robustez del diseño antes de proceder al layout.

### 4.3.2. Mejoras Específicas del ADC

#### Implementación de Bootstrap Switches

Una mejora significativa en la linealidad del ADC sería la implementación de switches bootstrapped según la técnica propuesta por Chang [7]. Estos switches deberían implementarse en:

- Switches de muestreo (Sample & Hold)
- Switches del CDAC que conectan a las entradas
- Potencialmente switches que conectan a  $V_{CM}$

La implementación de bootstrap mejoraría la resistencia ON constante a lo largo del rango de entrada, reduciendo la distorsión armónica total (THD) y mejorando el SFDR del conversor.

#### Comparación con Arquitectura de Top-Plate Sampling

Sería valioso comparar el desempeño de la arquitectura IMCS implementada con una arquitectura MCS estándar que utiliza top-plate sampling. Esta comparación permitiría cuantificar experimentalmente los beneficios de la insensibilidad a parásitos de la IMCS versus la simplicidad del muestreo top-plate.

### 4.3.3. Análisis Avanzados

#### Caracterización con Entradas Aleatorias

Aunque no es posible simular mismatch directamente, se podrían realizar simulaciones con entradas aleatorias para obtener métricas que se ven degradadas por efectos dinámicos:

- Offset dinámico del comparador
- Efectos de kickback sobre el CDAC

- Settling errors bajo diferentes condiciones de entrada
- Variación del tiempo de conversión con la amplitud de entrada

Este análisis proveería información valiosa sobre el comportamiento real del ADC en condiciones operativas variadas.

### **Análisis de Parásitos por Bloque**

Un estudio sistemático de los efectos de parásitos individuales por bloque sería extremadamente útil para guiar el proceso de layout. Esto incluiría:

- Degradación de la performance del comparador por parásitos en nodos críticos
- Impacto de resistencias serie en el CDAC sobre el settling time
- Efecto de capacitancias parásitas en el ancho de banda de buffers
- Sensibilidad del generador de pulsos a variaciones de parásitos

Este análisis permitiría priorizar esfuerzos de optimización de layout en los nodos más críticos.

### **4.3.4. Mejoras de Layout**

#### **Layout con Common-Centroid**

Un buen punto de partida para mejorar el matching sería implementar el layout del comparador utilizando técnicas de common-centroid para los pares diferenciales. Esto reduciría el offset sistemático y mejoraría la simetría del diseño, sirviendo como ejemplo didáctico de técnicas avanzadas de layout.

El CDAC también se beneficiaría enormemente de un layout common-centroid para los arrays de capacitores, mejorando el matching entre capacitores unitarios y reduciendo errores de INL/DNL.

#### **Completar Layout del Sistema Integrado**

La finalización del layout completo y la realización de simulaciones post-layout con parásitos extraídos es el siguiente paso crítico. Esto permitirá:

- Validar que el diseño mantiene funcionalidad con parásitos reales
- Cuantificar la degradación de performance por efectos de layout
- Identificar problemas de timing que no son visibles en simulaciones esquemáticas
- Completar la caracterización DNL/INL del sistema

### **4.3.5. Escalabilidad: Arquitectura Time-Interleaved**

Un objetivo de largo plazo para este diseño es su uso como canal individual en una arquitectura time-interleaved. Este trabajo fue concebido como el primer paso hacia un ADC multi-canal de alta velocidad.

## Motivación

La arquitectura time-interleaved permite escalar la velocidad de muestreo efectiva multiplicando el número de canales. Si este diseño alcanza 1 MSa/s por canal, un sistema de  $N$  canales intercalados podría alcanzar  $N$  MSa/s, extendiendo significativamente el rango de aplicaciones.

## Desafíos

La implementación time-interleaved introduce nuevos desafíos:

- **Offset mismatch:** Diferencias de offset entre canales generan espurios en el espectro de salida
- **Gain mismatch:** Diferencias de ganancia entre canales degradan SFDR
- **Timing skew:** Errores en el timing de muestreo entre canales limitan el SNDR a altas frecuencias de entrada
- **Complejidad de calibración:** Se requieren algoritmos de calibración digital para corregir mismatches

## Trabajo Futuro

Una vez completado y caracterizado este diseño de canal único, el trabajo futuro incluiría:

1. Optimización del canal único para minimizar área y consumo (facilitando la replicación)
2. Diseño del sistema de generación de fases de reloj con bajo skew
3. Implementación de múltiples canales con layout matched
4. Desarrollo de algoritmos de calibración digital para corrección de offset, ganancia y timing
5. Caracterización completa del sistema time-interleaved (SNDR vs frecuencia de entrada, SFDR, espurios por mismatch)

Este trabajo sienta las bases para ese desarrollo futuro, proveyendo un diseño funcional, documentado y verificado que puede servir como punto de partida.

## 4.4. Reflexión Final

Este proyecto demuestra que es posible diseñar circuitos integrados analógicos complejos utilizando exclusivamente herramientas de código abierto y PDKs accesibles. Si bien el camino presentó desafíos metodológicos y técnicos, el resultado es un diseño funcional documentado que puede servir como material educativo y punto de partida para desarrollos futuros.

La disponibilidad completa del código, esquemáticos, scripts y documentación en el repositorio público contribuye al ecosistema de diseño de código abierto y facilita que otros estudiantes e investigadores puedan replicar, mejorar y extender este trabajo.

# Bibliografía

- [1] J. G. D. Avila, “SAR ADC IHP Repository,” GitHub, 2025. [Online]. Disponible: [https://github.com/gsuckz/SAR\\_ADC\\_IHP](https://github.com/gsuckz/SAR_ADC_IHP)
- [2] X. Tang et al., “Low-Power SAR ADC Design: Overview and Survey of State-of-the-Art Techniques,” *IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS—I: REGULAR PAPERS*, VOL. 69, NO. 6, JUNE 2022.
- [3] V. Tripathi and B. Murmann, “An 8-bit 450-MS/s single-bit/cycle SAR ADC in 65-nm CMOS,” 2013 Proceedings of the ESSCIRC (ESSCIRC), Bucharest, Romania, 2013, pp. 117-120, doi: 10.1109/ESSCIRC.2013.6649086.
- [4] M. van Elzakker et al., “A 10-bit charge-redistribution ADC consuming  $1.9 \mu W$  at 1 MS/s,” *IEEE J. Solid-State Circuits*, vol. 45, no. 5, pp. 1007–1015, May 2010.
- [5] H. S. Bindra et al., “A 1.2-V dynamic bias latch-type comparator in 65-nm CMOS with 0.4-mV input noise,” *IEEE J. Solid-State Circuits*, vol. 53, no. 7, pp. 1902–1912, Jul. 2018.
- [6] T. Iizuka, R. Takenaka, H. Xu and A. A. Abidi, “Systematic Equation-Based Design of a 10-Bit, 500-MS/s Single-Channel SAR A/D Converter With 2-GHz Resolution Bandwidth,” in *IEEE Open Journal of the Solid-State Circuits Society*, vol. 4, pp. 147-162, 2024, doi: 10.1109/OJSSCS.2024.3469109
- [7] A. H. T. Chang, “Low-Power High-Performance SAR ADC with Redundancy and Digital Background Calibration,” Master’s thesis, Massachusetts Institute of Technology, 2013.
- [8] IHP GmbH, “SG13G2 Open Source Layout Rules,” *IHP-Open-PDK GitHub Repository*. [Online]. Disponible: [https://github.com/IHP-GmbH/IHP-Open-PDK/blob/main/ihp-sg13g2/libs.doc/doc/SG13G2\\_os\\_layout\\_rules.pdf](https://github.com/IHP-GmbH/IHP-Open-PDK/blob/main/ihp-sg13g2/libs.doc/doc/SG13G2_os_layout_rules.pdf)
- [9] An open-source adaptive event-based ADC for bio-signal acquisition in 130nm CMOS Dorrer, Simon [VerfasserIn]; 2025
- [10] The Relationship Between Rise Time and Bandwidth in Digital Signals *All About Circuits*. [Online]. Disponible: <https://www.allaboutcircuits.com/technical-articles/relationship-between-rise-time-bandwidth-digital-signal>
- [11] J. M. Rabaey, A. P. Chandrakasan y B. Nikolić, *Digital Integrated Circuits: A Design Perspective*, 2.<sup>a</sup> ed. Upper Saddle River, NJ: Prentice Hall, 2003.